

文件编号：02

项目名：外卖管理系统

需求与功能设计规格书

作者： 贾孟祺 丁雨晨 李艺涵 王烜铤

部门： 东大龙龙

时间： 2026/6/9

软件工程实验

改版履历

版数	年月日	改版内容	承认	查阅	作成	备注
V1.0	2026/6/7	初版（模板）			贾孟祺 丁雨晨 李艺涵 王烜铤	
V1.1	2026/6/9	完成功能模块详细设计			贾孟祺 丁雨晨 李艺涵 王烜铤	

目录

- 1 前言
 - 1.1 输入文档
 - 1.2 相关文档
 - 1.3 用语/术语定义
- 2 开发概要
 - 2.1 开发的背景和目的
 - 2.2 工作&运行环境
 - 2.3 系统构成
 - 2.4 功能整体图
 - 2.5 用户分类和特征
- 3 外部接口需求
 - 3.1 用户接口
 - 3.2 硬件接口
 - 3.3 软件接口
 - 3.4 通讯接口

4 实现方式

4.1 多端统一认证功能

4.2 顾客下单功能

4.3 商家管理功能

4.4 骑手配送功能

4.5 WebSocket实时推送功能

4.6 数据库层功能

5 非功能需求

5.1 权限需求

5.2 兼容互换性

5.3 安全性

5.4 健壮性

5.5 使用性（操作性）

5.6 效率性（性能）

5.7 维护性

5.8 移植性

5.9 用户文档

5.10 其他

6 使用和操作方法

6.1 环境设定

6.2 使用方法

7 注意限制事项

7.1 制约／限制事项

7.2 假定事项

7.3 特记事项

8 附录

8.1 业务规则

8.2 术语表

8.3 数据流图

1 前言

本文档为"全栈外卖管理系统（Takeaway Management System）"的需求与功能设计规格书，基于项目开发计划书和项目源代码实现进行功能模块分析与详细设计说明。

本系统是一套基于 B/S 架构的外卖管理平台，覆盖顾客下单、商家接单管理、骑手抢单配送的完整业务闭环，实现外卖业务流程的数字化、自动化和实时化。

1.1 输入文档

编号	输入项	内容描述	来源
IN-01	项目开发计划书	全栈外卖管理系统开发计划（含技术栈、模块结构、数据库设计、业务流转逻辑等5大章节需求）	项目团队
IN-02	项目源代码	后端 FastAPI 工程 + 前端 Vue 3 工程	项目团队
IN-03	01-Plan.pdf	软件项目开发计划书模板	课程提供

1.2 相关文档

编号	文档名	说明
DOC-01	项目开发计划书.md	项目整体计划、人员分工、技术方案、开发日程
DOC-02	README.md	部署说明文档（含环境要求、安装步骤、使用指南）
DOC-03	02_FDesign (1).md	需求与功能设计规格书模板

1.3 用语/术语定义

术语	说明
Customer（顾客）	系统的消费者角色，可浏览菜品、管理收货地址、提交订单、查看历史订单
Merchant（商家）	系统的餐厅/店主角色，可管理菜品（增删改查/上下架）、处理订单（接单/拒单/指派骑手）
Courier（骑手）	系统的配送员角色，可查看可抢订单、主动抢单、确认配送完成
JWT	JSON Web Token，基于 JSON 的无状态身份认证令牌，HS256 算法签名
RBAC	Role-Based Access Control，基于角色的访问控制（customer / merchant / courier）
SPA	Single Page Application，单页面应用，前端路由切换无整页刷新
ORM	Object-Relational Mapping，对象关系映射，将数据库表映射为 Python 对象

术语	说明
WebSocket	基于 TCP 的全双工实时通信协议，用于服务端向客户端实时推送消息
软删除	通过设置 <code>is_deleted=1</code> 标记逻辑删除，数据物理保留，不影响历史订单外键
事务（Transaction）	数据库操作的原子性保证：多个操作要么全部成功，要么全部回滚（ROLLBACK）
状态机（State Machine）	订单状态的有序流转规则模型（0→1→2→3 / 0→4 / 0→5）
bcrypt	密码单向哈希算法，将用户密码加密为不可逆的哈希值存储
CORS	Cross-Origin Resource Sharing，跨域资源共享，允许前端跨端口访问后端 API

2 开发概要

2.1 开发的目的和背景

开发目的：

本项目旨在构建一套面向餐饮外卖行业的全栈信息化管理平台，覆盖顾客下单、商家接单管理、骑手抢单配送的完整业务闭环，实现外卖业务流程的数字化、自动化和实时化。

开发背景：

随着移动互联网和电子商务的快速发展，外卖配送已成为居民日常消费的重要场景。据行业数据显示，中国外卖市场规模已突破万亿元，用户规模超过5亿。然而，大量中小型餐饮商家仍依赖人工电话接单、纸质记录、口头指派骑手等低效方式运营，存在以下痛点：

- 订单信息传递延迟，顾客等待时间不可预估
- 商家与骑手之间缺乏高效的协同机制
- 订单状态不透明，顾客无法实时了解配送进展
- 软删除与历史数据追溯困难

本系统从零开发一套完整的外卖管理系统，针对性地解决上述痛点，实现：

- 顾客在线浏览菜品并完成下单
- 商家实时接收订单提醒并处理订单
- 骑手主动抢单并完成配送确认
- 全流程订单状态实时同步与完整追溯

必要性与可行性：

- **经济可行性：**系统采用开源技术栈（Python/FastAPI + Vue 3），开发成本低，后期维护成本可控
- **技术可行性：**前后端分离架构成熟，WebSocket 实时通信技术已在工业界广泛验证
- **法规合规：**系统遵循《网络安全法》《个人信息保护法》相关要求，密码加密存储，JWT 无状态认证保障用户数据安全
- **道德准则：**公平对待顾客、商家、骑手三方利益，不设置算法歧视机制，骑手抢单机制公开透明

预期目标市场：

本系统定位于中小型餐饮商家及区域性外卖配送服务商，产品原型计划于 2026 年 6 月完成并进入内部测试阶段。

2.2 工作&运行环境

2.2.1 硬件环境

项目	最低配置	推荐配置
CPU	Intel Core i5 / AMD Ryzen 5	Intel Core i7 / AMD Ryzen 7
内存	8 GB DDR4	16 GB DDR4
磁盘	256 GB SSD	512 GB SSD
网络	100 Mbps LAN	1000 Mbps LAN
显示器	1920×1080	2560×1440 双屏

2.2.2 软件环境

本节明确系统功能能够运行所必需的操作系统、浏览器及相关软件，按**产品运行依赖**（系统运行时必须安装/具备的软件）与**开发构建依赖**（仅在开发、构建阶段需要的工具）分别列出。

2.2.2.1 操作系统

OS	最低版本	说明
Windows	10 / 11 (64-bit)	开发与部署均可
macOS	12+	开发与部署均可
Linux	Ubuntu 20.04+ / CentOS 8+	推荐生产部署

Python 3.10+ 与 Node.js 18+ 均为跨平台运行时，源码无需修改即可在上述 OS 上运行。

2.2.2.2 Web 浏览器

本系统为 B/S 架构 Web 应用，浏览器是用户端运行的必要条件。前端依赖现代浏览器 API（ES Module、WebSocket、CSS Grid/Flexbox）。

浏览器	最低版本	说明
Google Chrome	90+	★ 首选，Vue Devtools 调试体验最佳
Microsoft Edge	90+（Chromium 内核）	★ 推荐，与 Chrome 同等兼容
Mozilla Firefox	90+	可用，部分 CSS 有细微差异
Apple Safari	15+	macOS / iOS 可用

不支持： Internet Explorer（所有版本）、国产浏览器「兼容模式」（IE 内核）、文本浏览器。

各端推荐分辨率与布局：

端	布局宽度	推荐分辨率	适用场景
顾客端	max-width: 600px 居中	375×812	模拟移动端购物体验
骑手端	max-width: 600px 居中	375×812	移动端抢单与配送操作
商家端	max-width: 1100px	1920×1080	PC 工作台（表格/表单密集操作）
登录页	自适应居中卡片	全适配	手机/平板/PC 均可

浏览器技术依赖清单：

浏览器 API	使用位置
ES Module (import/export)	Vite 构建产物，模块化加载
WebSocket	商家端/骑手端实时推送
localStorage	JWT Token 持久化
Fetch API	Axios 底层 HTTP 请求
CSS Grid + Flexbox	Element Plus 组件布局

2.2.2.3 产品运行依赖

以下为系统部署运行**必须安装**的软件及软件包。缺失任何一项将导致系统无法正常启动或功能异常。

(1) Server 端运行时

类别	软件/包	最低版本	说明
语言运行时	Python	3.10+	后端代码执行环境
数据库	SQLite	3.x	开发环境，零配置
数据库	MySQL	8.0+	生产环境（开发环境无需安装）

Python 依赖包（[requirements.txt](#)，均通过 [pip install](#) 安装）：

包名	版本	用途
fastapi	0.115.0	Web 框架（路由/依赖注入/自动文档）
uvicorn	0.30.0	ASGI 服务器（HTTP + WebSocket 入口）
sqlalchemy	2.0.35	异步 ORM，屏蔽 SQL 方言差异
aiosqlite	0.20.0	SQLite 异步驱动（开发环境）
asyncmy	0.2.9	MySQL 异步驱动（生产环境）

包名	版本	用途
python-jose	3.3.0	JWT 签发与验证 (HS256)
passlib	1.7.4	bcrypt 密码单向哈希
python-multipart	0.0.12	表单数据解析
pydantic	2.9.0	请求/响应数据校验
pydantic-settings	2.5.0	环境变量与 .env 配置管理
websockets	13.0	WebSocket 长连接协议支持

以上 11 个 Python 包均为**产品运行依赖**，Server 启动时必须全部已安装。

(2) 客户端运行时（浏览器端）

客户端为 Vue 3 SPA，所有前端框架及库经 Vite 构建打包为纯静态文件（HTML/CSS/JS），部署于 Web 服务器（Nginx / Apache / Vite Dev Server）。用户浏览器无需安装任何插件。

前端框架/库	版本	运行时角色
Vue 3	3.4.0	MVVM 响应式框架，驱动页面渲染
vue-router	4.3.0	SPA 前端路由（Hash 模式）
Pinia	2.1.0	全局状态管理（Token/用户信息）
Element Plus	2.7.0	UI 组件库（表单/表格/弹窗/通知）
@element-plus/icons-vue	2.3.0	UI 图标集
Axios	1.7.0	HTTP 请求 + JWT 拦截器 + 401 处理

以上前端库在构建时（`npm run build`）被打包进 `dist/` 目录的静态产物中，随 HTML 一同下发至浏览器执行。Server 不依赖这些包。

客户端运行时关键行为：

行为	实现机制
SPA 页面切换	vue-router Hash 模式，无整页刷新
Token 持久化	Pinia → localStorage (<code>token/role/user_id/user_name</code>)
请求鉴权	Axios 拦截器自动附加 <code>Authorization: Bearer <token></code>
Token 过期	响应拦截器 401 → 清除 localStorage → 重定向 /
路由守卫	<code>router.beforeEach</code> 校验 <code>meta.role</code> ，未登录/角色错跳转
实时推送	WebSocket → <code>onmessage</code> → ElNotification 弹窗 + 声音提示
错误展示	API <code>message</code> 字段 → ElMessage 组件
操作确认	关键操作 → ElMessageBox 确认对话框

行为	实现机制
断线恢复	<code>ws.onclose</code> 5 秒后自动重连

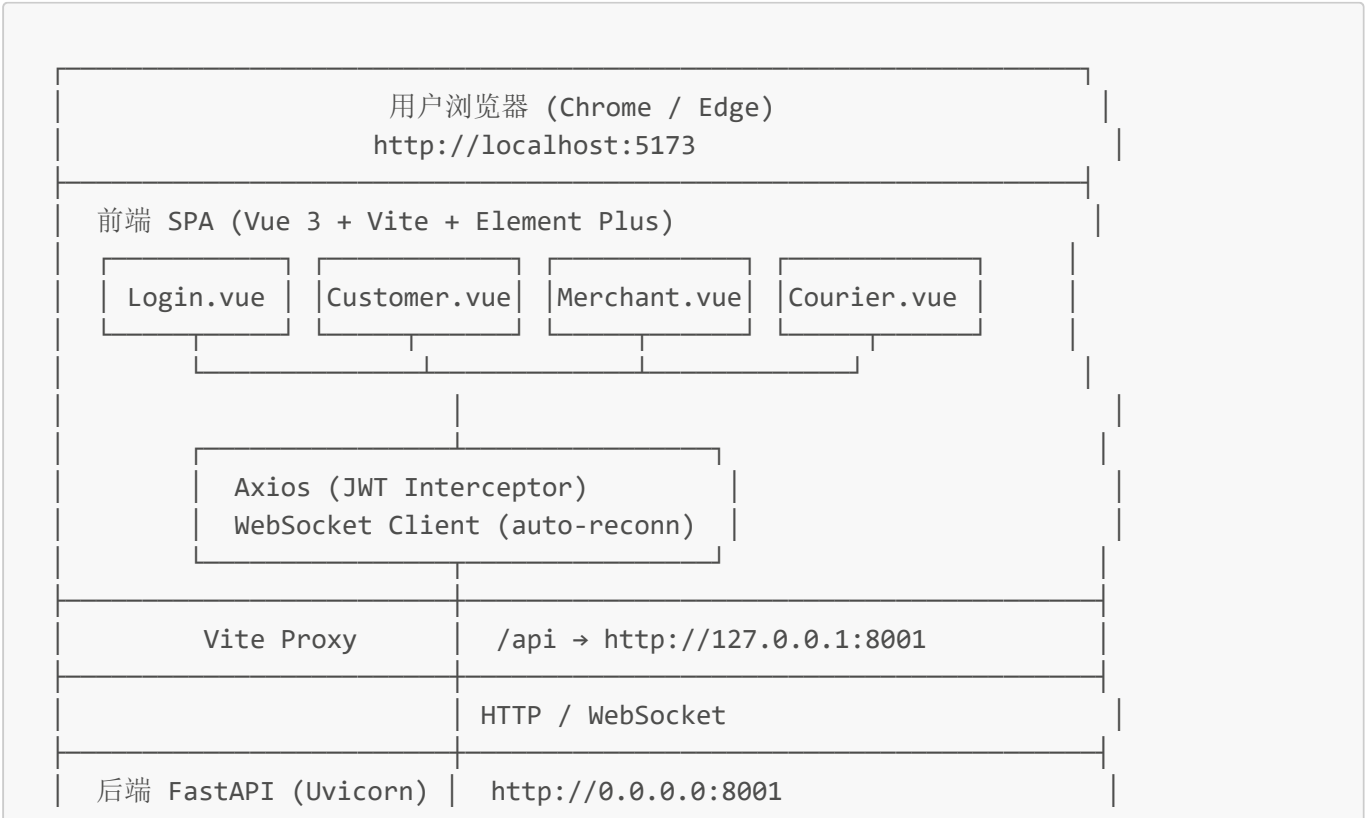
2.2.2.4 开发构建依赖

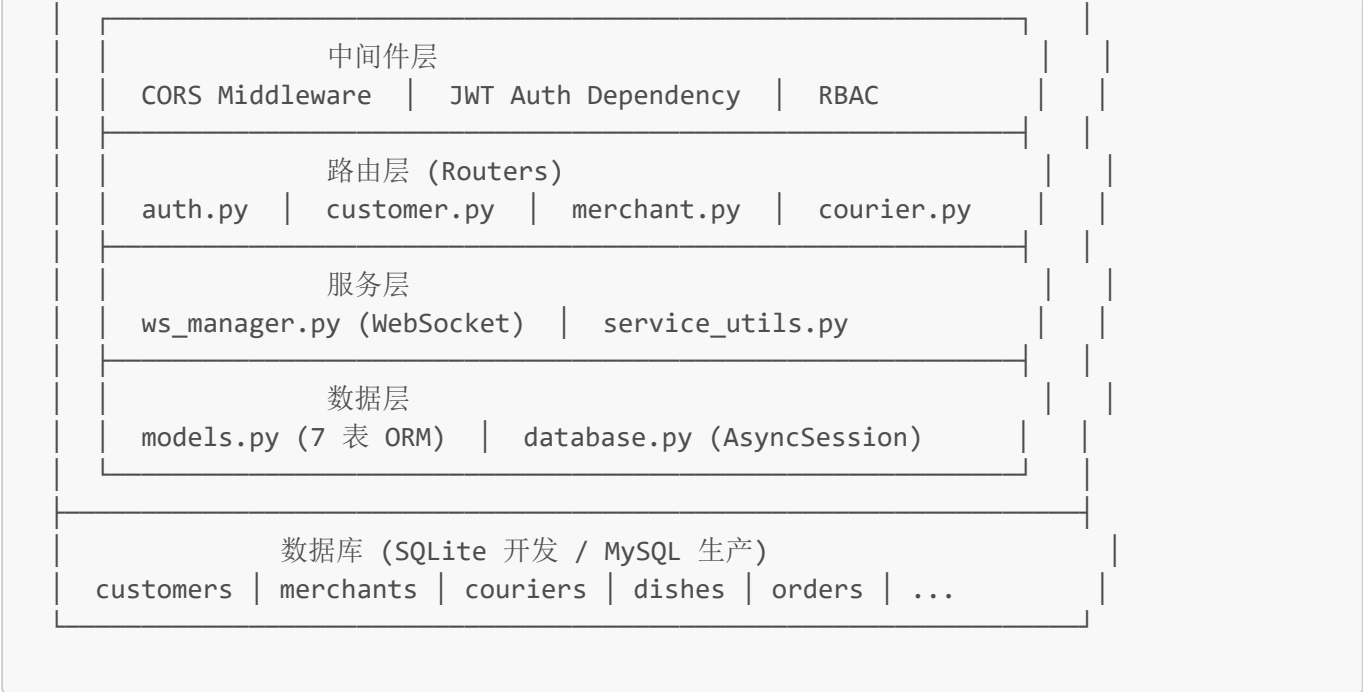
以下工具仅在**开发、构建阶段**需要，产品运行时**不需要**安装或运行。

软件/包	最低版本	用途	备注
Node.js	18+ LTS	JS 运行时（Vite 开发服务器 + 生产构建）	开发机安装即可
npm	9+	前端包管理器	随 Node.js 一同安装
Vite	5.4.0	开发服务器（HMR）+ 生产构建工具	<code>npm run dev</code> / <code>npm run build</code>
@vitejs/plugin-vue	5.0.0	Vite Vue 3 SFC 编译插件	构建阶段使用
VS Code	最新版	推荐 IDE（非强制）	可用任意编辑器替代
Git	2.40+	版本控制（非强制）	仅团队协作开发需要

注意：生产部署时，前端只需将 `npm run build` 生成的 `dist/` 目录下的静态文件交由 Web 服务器（Nginx 等）托管即可。Node.js、npm、Vite 等构建工具链在生产服务器上不是必需的。

2.2.3 系统部署图



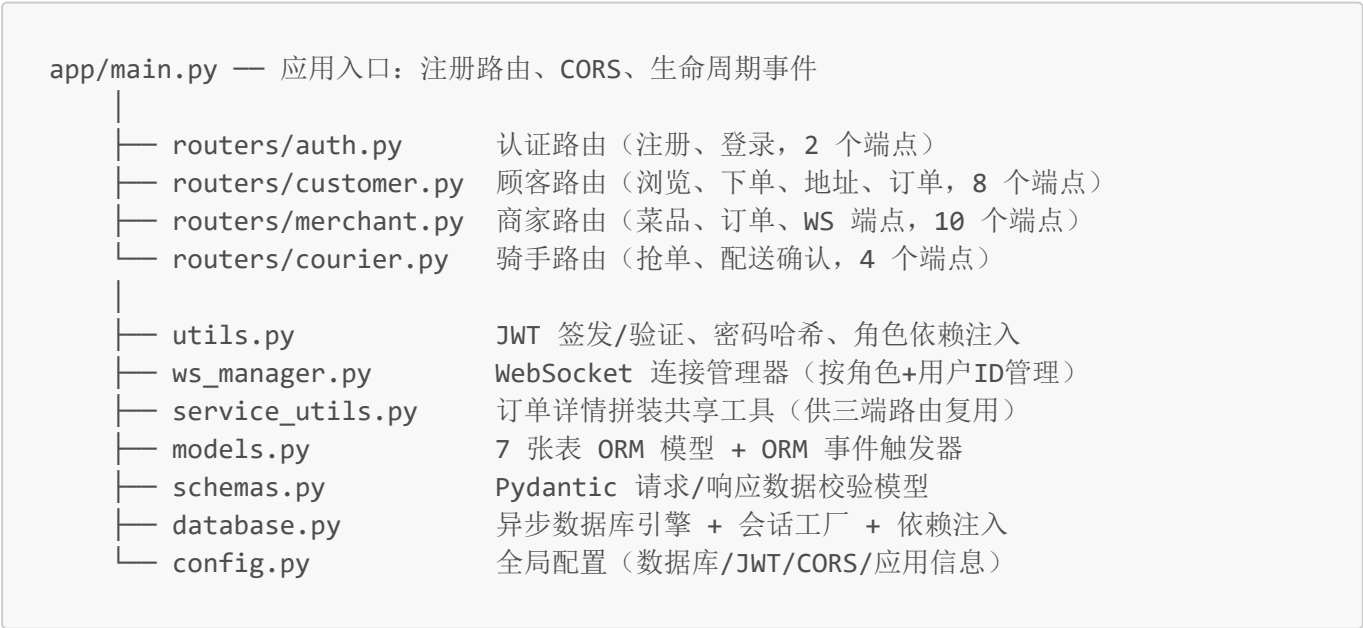


2.3 系统构成

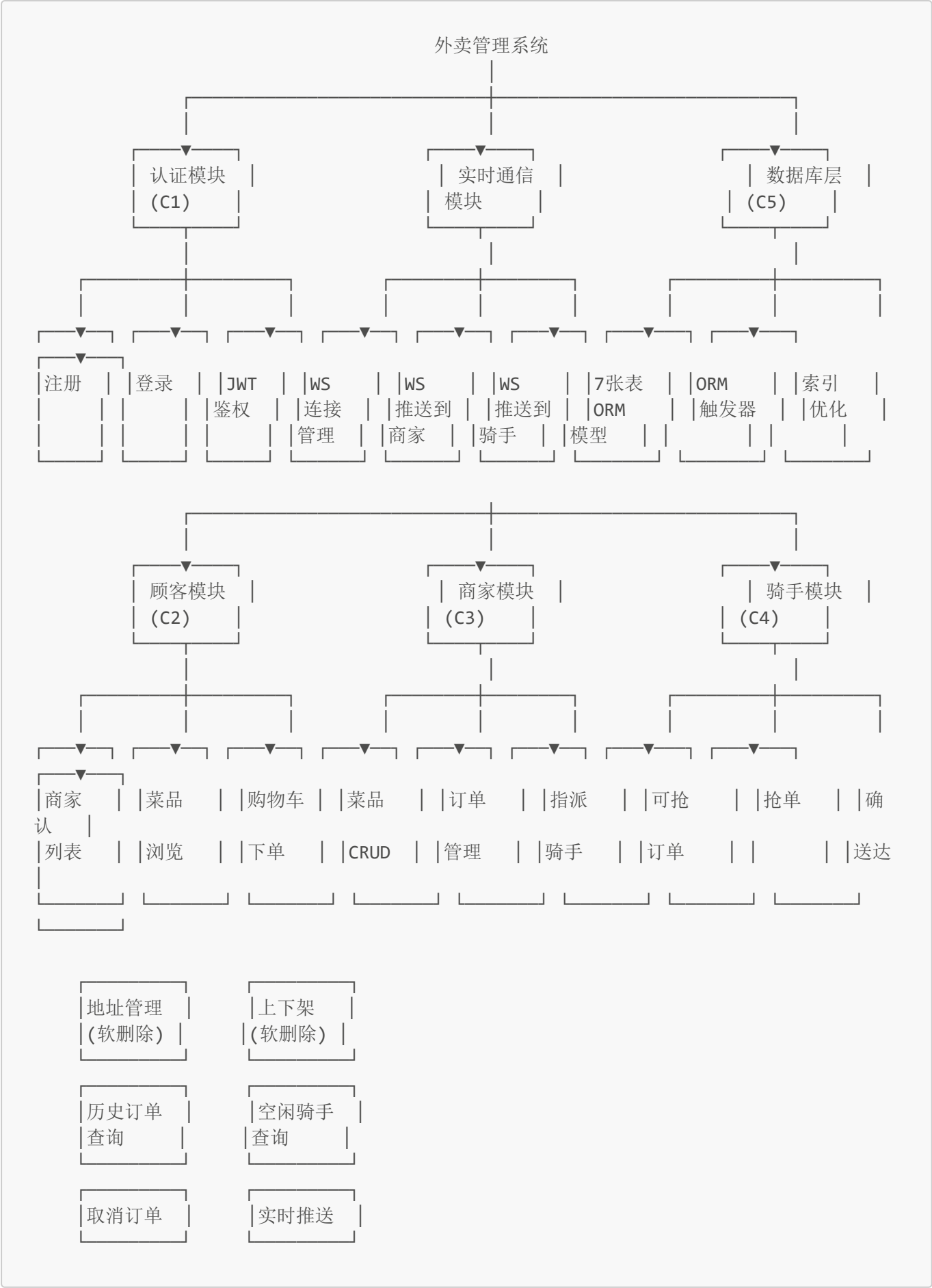
本系统采用 B/S 架构（浏览器/服务器模式），基于前后端分离设计，划分为三层：

层级	技术栈	职责
表现层	Vue 3 + Element Plus + Pinia	用户界面：登录/注册、顾客端（移动端）、商家端（PC）、骑手端（移动端）
服务层	FastAPI + SQLAlchemy + WebSocket	认证鉴权、业务逻辑（状态机流转）、事务管理、WebSocket 实时推送
数据层	SQLite（开发）/ MySQL（生产）	数据持久化存储，7 张核心业务表

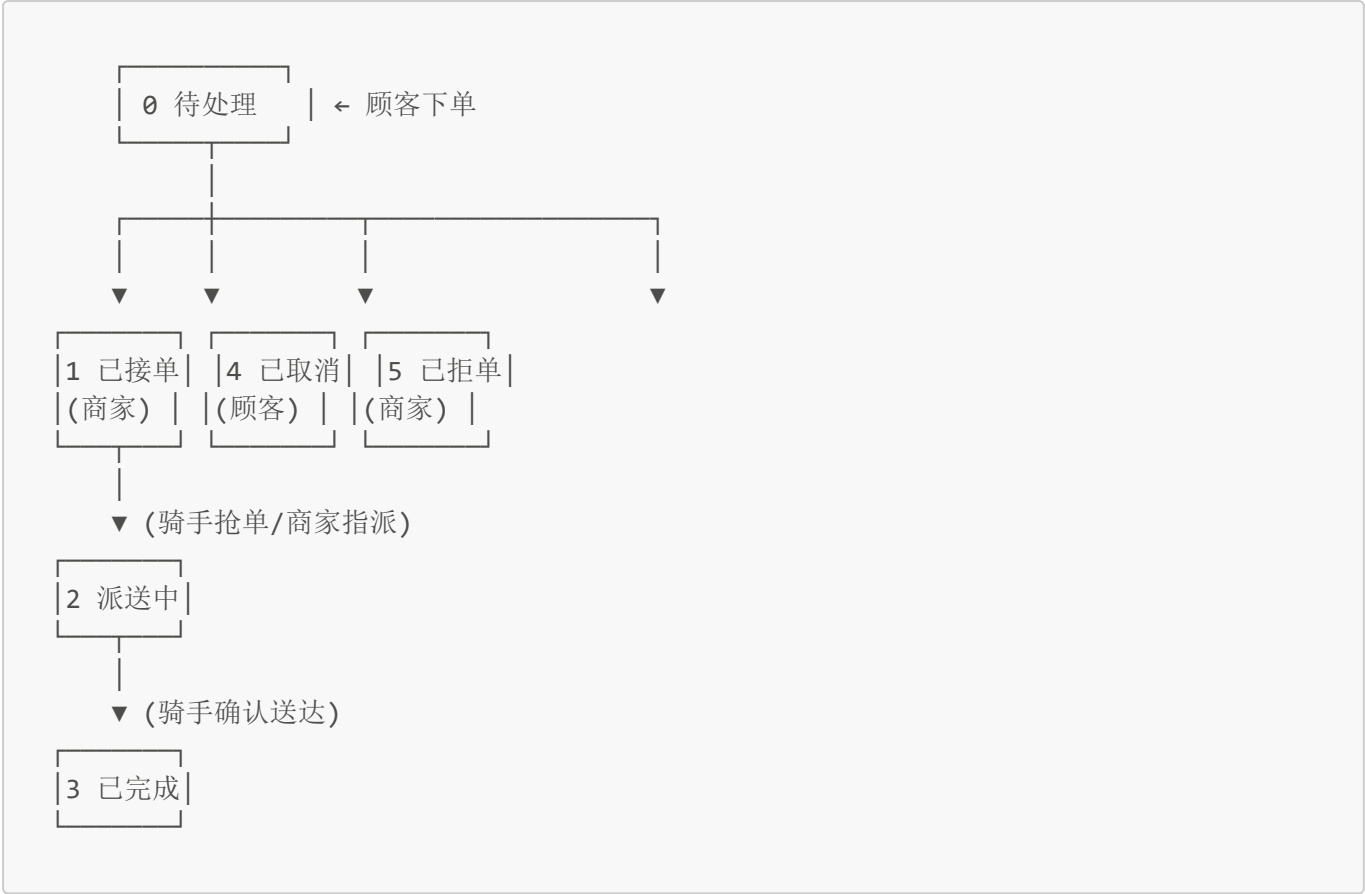
系统组件结构及关联关系：



2.4 功能整体图



订单状态机:



2.5 用户分类和特征

用户一览表：

NO	用户	别名	描述
1	顾客 (Customer)	消费者	浏览菜品、管理收货地址、购物车下单、查看历史订单、取消待处理订单
2	商家 (Merchant)	餐厅/店主	菜品管理（增删改查/上下架/软删除）、订单处理（接单/拒单/指派骑手）、空闲骑手查询
3	骑手 (Courier)	配送员	查看可抢订单列表、主动抢单、查看我的配送订单、确认送达完成配送

各角色的职责：

- 1. **顾客 (Customer)**：通过移动端页面浏览商家在售菜品（系统自动过滤已下架和已删除菜品）、管理个人收货地址（支持软删除，删除后不影响历史订单）、添加菜品至购物车并提交订单（系统采用原子事务保证数据一致性）、查看历史订单记录、取消待处理状态的订单。
- 2. **商家 (Merchant)**：通过 PC 工作台管理菜品全生命周期——新增、编辑（名称/价格/上下架状态）、软删除；实时查看订单列表并支持按状态筛选；处理订单：接单（0→1）或拒单（0→5）；查询当前空闲骑手列表并指派配送任务（1→2）；通过 WebSocket 实时接收新订单推送通知。
- 3. **骑手 (Courier)**：通过移动端页面查看商家已接单但尚无骑手配送的"可抢订单"列表；主动发起抢单操作（1→2，同时骑手状态由空闲变为忙碌）；查看已分配给自己的配送订单列表；配送完成后确认送达（2→3，同时骑手状态释放为空闲）；通过 WebSocket 实时接收新任务指派推送。

功能模块与用户的对应一览表：

功能模块	顾客	商家	骑手
用户注册	✓	✓	✓
用户登录	✓	✓	✓
JWT 鉴权与 RBAC	✓	✓	✓
商家列表查询	✓		
菜品浏览（过滤已下架/已删除）	✓		
收货地址 CRUD（软删除）	✓		
购物车下单（原子事务）	✓		
历史订单查询	✓		
取消待处理订单	✓		
菜品 CRUD（添加/编辑/软删除）		✓	
菜品上下架		✓	
订单列表与状态筛选		✓	
接单/拒单操作		✓	
指派骑手		✓	
空闲骑手查询		✓	
订单查询	✓	✓	✓
可抢订单列表			✓
主动抢单			✓
我的配送订单			✓
确认送达			✓
WebSocket 实时推送接收		✓	✓

3 外部接口需求

3.1 用户接口

3.1.1 用户输入数据

用户类型	输入数据	接口形式
顾客	用户名、密码（注册/登录）；收货地址（地址文本+电话号码）；菜品选择与数量；订单取消确认	Web 表单、按钮操作、下拉选择、数量调整
商家	用户名、密码（注册/登录）；菜品信息（名称+价格）；接单/拒单确认；骑手指派选择	Web 表单、按钮操作、下拉选择
骑手	用户名、密码（注册/登录）；抢单确认；送达确认	Web 表单、按钮操作

3.1.2 系统输出数据

输出类型	数据内容	展现形式
认证结果	JWT Token（access_token）、用户ID、角色类型、用户名	API Response JSON
商家列表	商家ID、商家名称、在售菜品数量	卡片网格
菜品列表	菜品ID、名称、价格、上架状态	列表卡片
订单信息	订单ID、顾客名、商家名、菜品摘要、总额、状态中文名、时间	卡片/表格列表
实时推送	新订单提醒（商家）、新任务指派（骑手）、订单取消提醒（商家）	浏览器通知弹窗 + 声音提醒
错误提示	业务校验失败原因（如"仅待处理订单可取消"、"该订单已被其他骑手抢走"）	Element Plus Message / 弹窗

3.2 硬件接口

本系统为纯 B/S 架构 Web 应用，不直接与硬件设备交互。系统运行在通用 x86 架构 PC 服务器上，用户通过标准浏览器（Chrome / Edge）访问。

接口名	描述
无	本系统无特殊硬件接口需求

3.3 软件接口

接口名	描述	技术实现
RESTful API 接口	前后端数据通信接口，统一 {code, message, data} 响应格式	HTTP/HTTPS，FastAPI 框架
WebSocket API	服务端向指定商家/骑手实时推送订单状态变更消息	WS 协议，websockets 库
Swagger UI	API 接口文档自动生成，支持在线交互式测试	FastAPI 内置 /docs

接口名	描述	技术实现
数据库接口	ORM 抽象层，避免手写 SQL 拼接	SQLAlchemy 2.0 异步模式
JWT 认证接口	Token 签发与验证，客户端通过 HTTPBearer 方式传递	python-jose 库
Vite 代理接口	开发环境前端自动将 /api 请求代理到后端	Vite proxy 配置

内部组件间数据交换：

组件间	交换数据	交换方式
前端 → 后端	认证凭证、CRUD 请求参数、订单数据	Axios HTTP + JWT Bearer Header
后端 → 前端	JSON 响应数据、WebSocket 推送消息	HTTP Response + WS JSON Message
路由层 → 数据层	SQLAlchemy ORM 查询/插入/更新/删除	AsyncSession 异步查询
路由层 → WebSocket 管理器	推送消息对象	<code>ws_manager.send_to_merchant()</code> / <code>send_to_courier()</code>
中间件层 → 路由层	已认证的用户 ORM 实例	FastAPI Depends 依赖注入

3.4 通讯接口

通信方式	协议	默认端口	用途
HTTP REST API	HTTP/1.1	8001	常规 CRUD 操作、认证请求
WebSocket	WS	8001	实时订单状态推送（商家端、骑手端）
Vite Dev Proxy	HTTP/1.1	5173	前端开发服务器代理 /api 请求到后端
MySQL TCP 连接	MySQL Wire Protocol	3306	生产环境数据库连接（开发环境使用本地 SQLite 文件）

WebSocket 消息格式：

```
{
  "type": "new_order | order_cancelled | order_assigned",
  "title": "消息标题（含 Emoji 图标）",
  "body": "消息正文",
  "order_id": 123,
  "timestamp": "2026-06-09T12:00:00"
}
```

WebSocket 消息推送场景：

消息类型	触发条件	推送目标	触发代码位置
new_order	顾客下单成功	对应商家	customer.py#L229
order_cancelled	顾客取消订单	对应商家	customer.py#L299
order_assigned	商家指派骑手	对应骑手	merchant.py#L244

WebSocket 连接鉴权：

- 连接端点：ws://host:8001/api/merchant/ws/{role}/{user_id}?token={jwt_token}
- 鉴权方式：通过 Query String 传递 JWT Token，服务端调用 parse_token_optional() 解析校验
- Token 验证失败 → 关闭连接，返回 code=4003（Token 无效或角色不匹配）
- 前端断线自动重连：5 秒后自动重新建立 WebSocket 连接

安全性：

- JWT Token 有效期 24 小时，过期需重新登录
- WebSocket 连接需附带有效 Token，中间人无法伪造
- 前端 401 拦截器在 Token 过期时自动清除登录状态并重定向

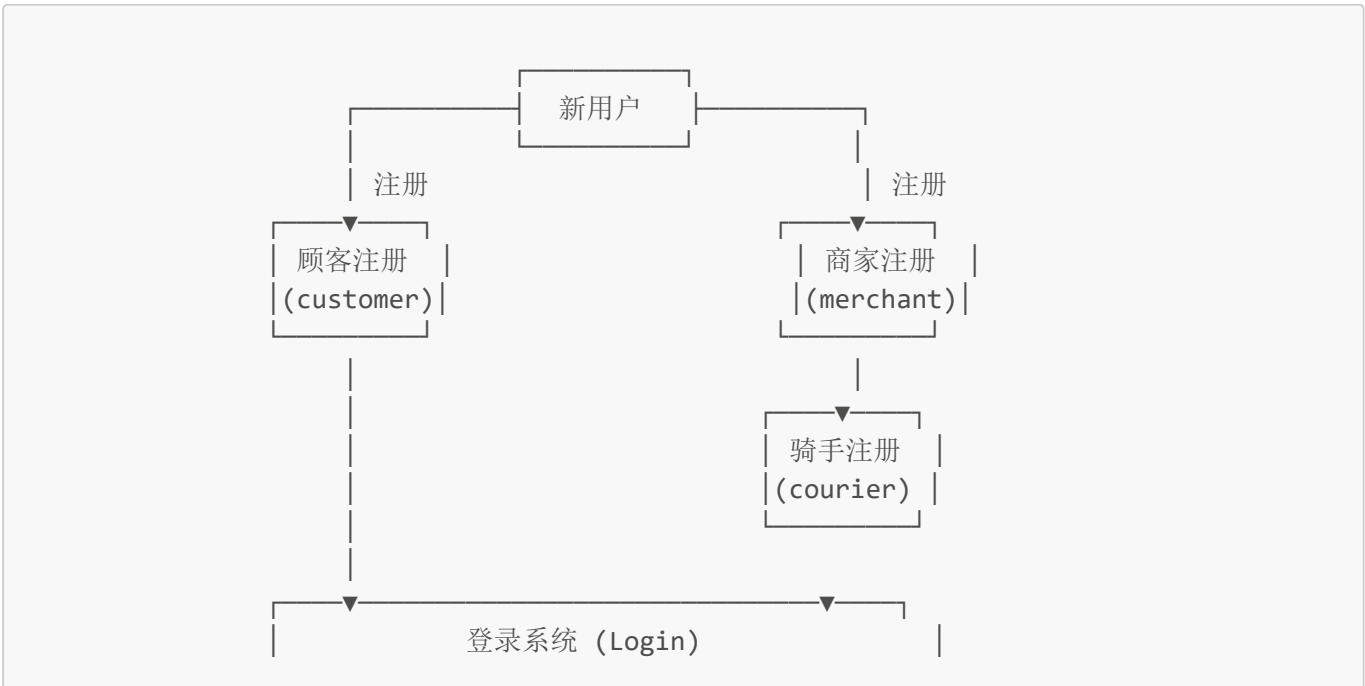
4 实现方式

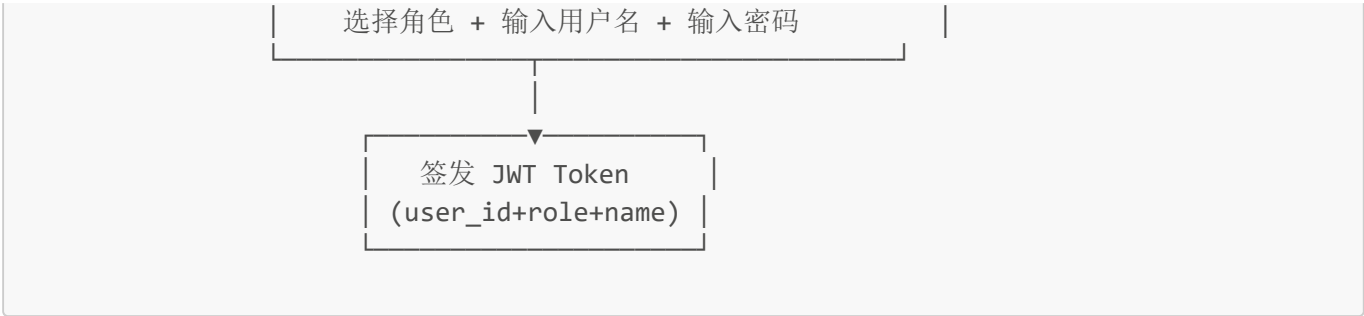
4.1 多端统一认证功能

4.1.1 功能概要

本功能模块实现顾客、商家、骑手三种角色在同一登录页面的统一注册与登录。系统根据用户选择的角色类型，将账户信息写入对应的数据库表（customers / merchants / couriers），并签发包含用户身份信息的 JWT Token。后端通过角色依赖注入确保各端 API 只能被对应角色的用户访问。

用例图：



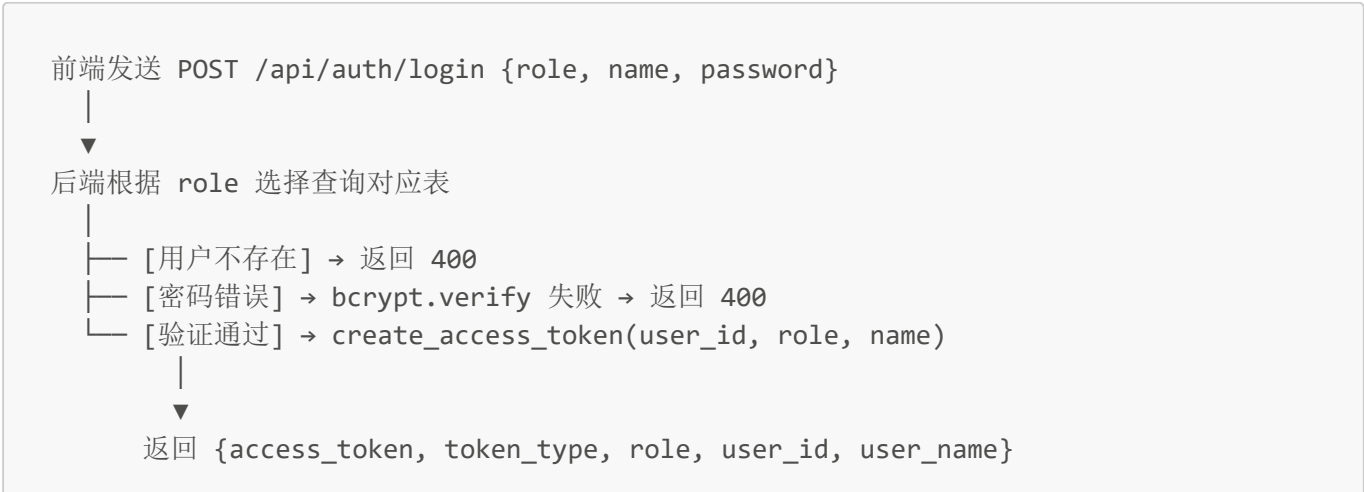


4.1.2 功能详细

注册流程（活动图）：



登录鉴权流程：





代码实现位置：

组件	文件	说明
注册/登录路由	routers/auth.py	2 个端点：POST /register、POST /login
JWT 签发	utils.py#L31-L40	<code>create_access_token()</code> ，24h 过期
密码哈希	utils.py#L22-L27	<code>hash_password()</code> / <code>verify_password()</code>
角色依赖注入	utils.py#L53-L101	<code>get_current_customer()</code> / <code>get_current_merchant()</code> / <code>get_current_courier()</code>
JWT 配置	config.py	SECRET / ALGORITHM / EXPIRE_MINUTES
登录/注册页面	Login.vue	角色选择 + 表单 + 登录/注册切换
认证状态管理	stores/auth.js	Pinia store，Token 持久化到 localStorage
Axios 拦截器	api/index.js	请求拦截器自动附带 Bearer Token；响应拦截器 401 自动跳转
路由守卫	router/index.js	beforeEach 校验 meta.role，未登录重定向，角色不匹配跳转

4.1.3 相关功能

组件名	说明	关系
所有业务路由	各端业务路由依赖认证中间件注入当前用户实例	所有端点均 <code>Depends(get_current_xxx)</code>
前端路由守卫	基于 Pinia auth store 的 token + role 控制页面访问	角色校验 + 未登录重定向到 /
WebSocket 端点	WS 连接时需通过 <code>parse_token_optional()</code> 解析 Token	Token 放在 Query String 中传递

4.1.4 用户接口

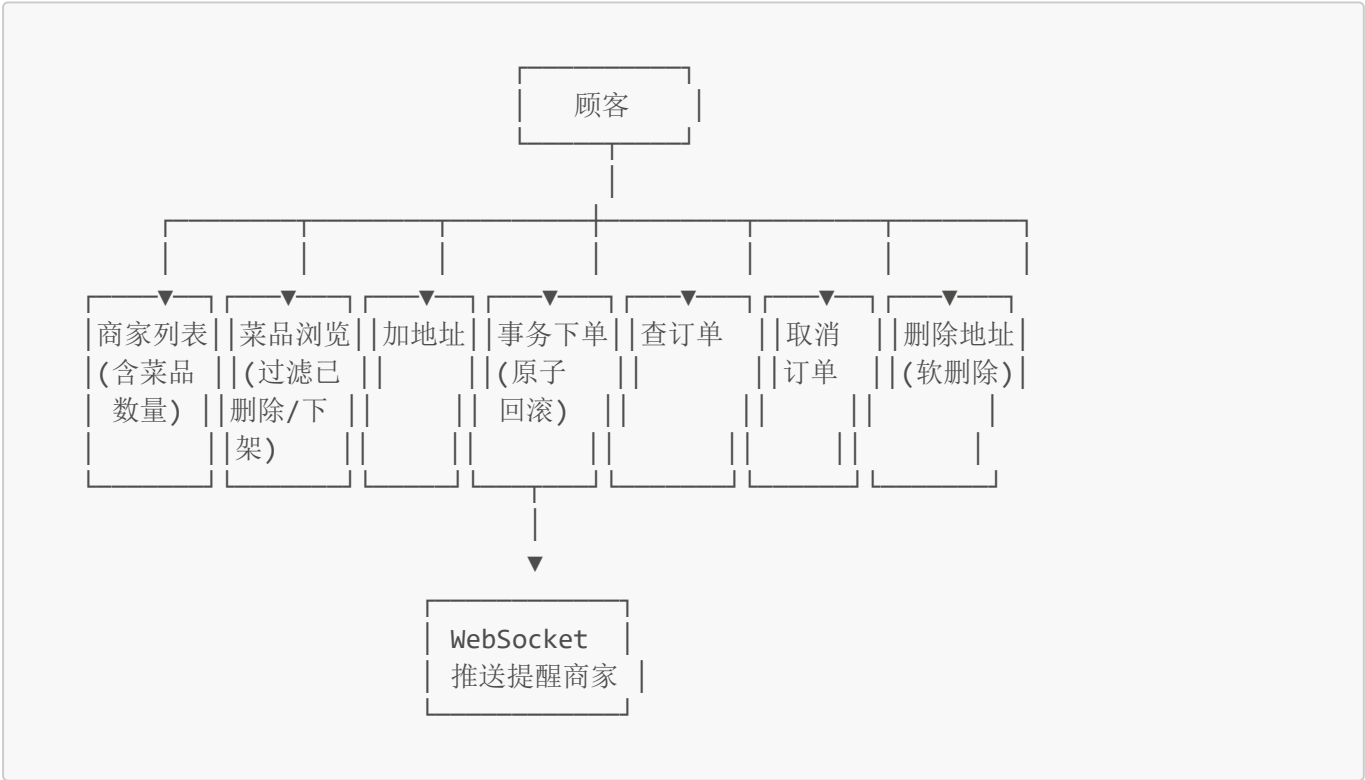
接口端点	方法	输入	输出
/api/auth/register	POST	{role: "customer" "merchant" "courier", name, password}	{code: 200, message: "注册成功", data: {user_id, name, role}}
/api/auth/login	POST	{role: "customer" "merchant" "courier", name, password}	{code: 200, message: "登录成功", data: {access_token, token_type, role, user_id, user_name}}

4.2 顾客下单功能

4.2.1 功能概要

顾客端实现从"浏览商家 → 选择菜品 → 管理地址 → 提交订单 → 查看订单 → 取消订单"的完整点餐流程。核心下单接口采用**显式异步事务 (async with db.begin())** 保证原子性：任何校验失败或运行时异常将触发自动 ROLLBACK，杜绝产生僵尸订单（仅有 Orders 主记录而无 OrderDish 明细的脏数据）。

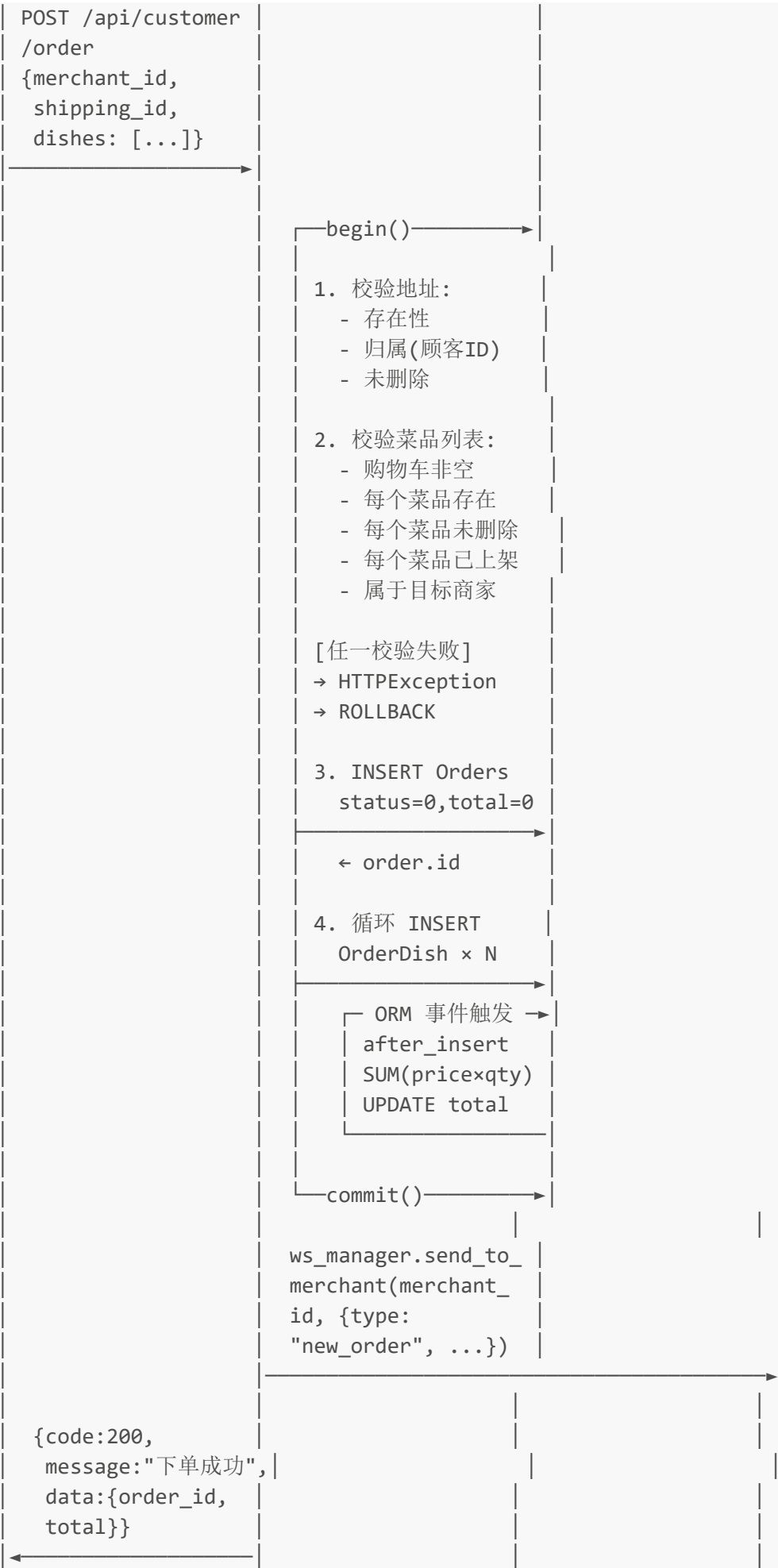
用例图：



4.2.2 功能详细

事务下单流程（顺序图）：





下单校验规则表：

校验项	条件	失败状态码	失败消息
收货地址存在性	<code>shipping != null</code>	404	"收货地址不存在"
地址归属	<code>shipping.customer_id == customer.id</code>	403	"收货地址不属于当前顾客"
地址未删除	<code>shipping.is_deleted == 0</code>	400	"收货地址已被删除"
购物车非空	<code>len(req.dishes) > 0</code>	400	"请至少选择一个菜品"
菜品存在性	<code>dish in dish_map</code>	404	"菜品 #{id} 不存在"
菜品未删除	<code>dish.is_deleted == 0</code>	400	"菜品「{name}」已被删除"
菜品已上架	<code>dish.is_active == 1</code>	400	"菜品「{name}」已下架"
菜品属于目标商家	<code>dish.merchant_id == req.merchant_id</code>	400	"菜品「{name}」不属于该商家"

取消订单流程：



代码实现位置：

组件	文件	说明
顾客路由	<code>routers/customer.py</code>	8 个端点：商家列表、菜品浏览、地址 CRUD(3)、事务下单、订单列表、取消订单
订单拼装	<code>service_utils.py</code>	<code>build_order_out()</code> 订单详情拼装（菜品摘要、明细、状态中文）
顾客页面	<code>Customer.vue</code>	5 个 Tab：商家、菜单、购物车、地址、订单
API 调用	<code>api/services.js</code>	顾客相关 9 个 API 封装函数

4.2.3 相关功能

组件名	说明	关系
WebSocket 管理器	下单成功后推送 new_order 提醒给对应商家；取消后推送 order_cancelled	<code>ws_manager.send_to_merchant()</code>
ORM 触发器	下单明细写入后自动触发总额计算	OrderDish after_insert 事件
商家路由	商家接单/拒单依赖于订单状态=0	状态机联动

4.2.4 用户接口

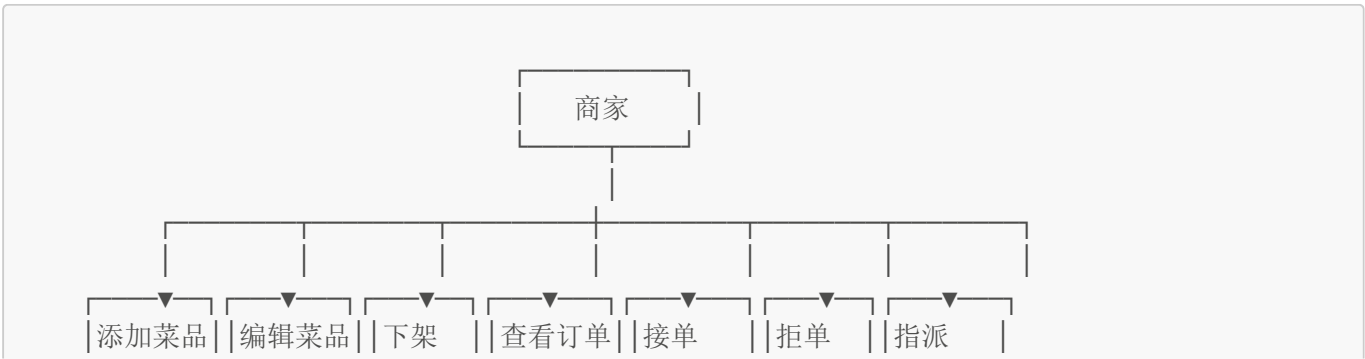
接口端点	方法	功能描述	鉴权角色
<code>/api/customer/merchants</code>	GET	获取有在售菜品的商家列表（含菜品数量统计）	customer
<code>/api/customer/dishes?merchant_id=</code>	GET	浏览菜品（仅展示 is_deleted=0 且 is_active=1）	customer
<code>/api/customer/addresses</code>	GET	获取当前顾客未删除的地址列表	customer
<code>/api/customer/addresses</code>	POST	新增收货地址	customer
<code>/api/customer/addresses/{id}</code>	DELETE	软删除地址（is_deleted=1，历史订单不受影响）	customer
<code>/api/customer/order</code>	POST	事务下单（原子化：地址校验→菜品校验→插入订单→插入明细→自动计算总额）	customer
<code>/api/customer/orders</code>	GET	查看历史订单（含菜品详情拼装，按时间倒序）	customer
<code>/api/customer/order/{id}/cancel</code>	PUT	取消待处理订单（0→4，状态机限制）	customer

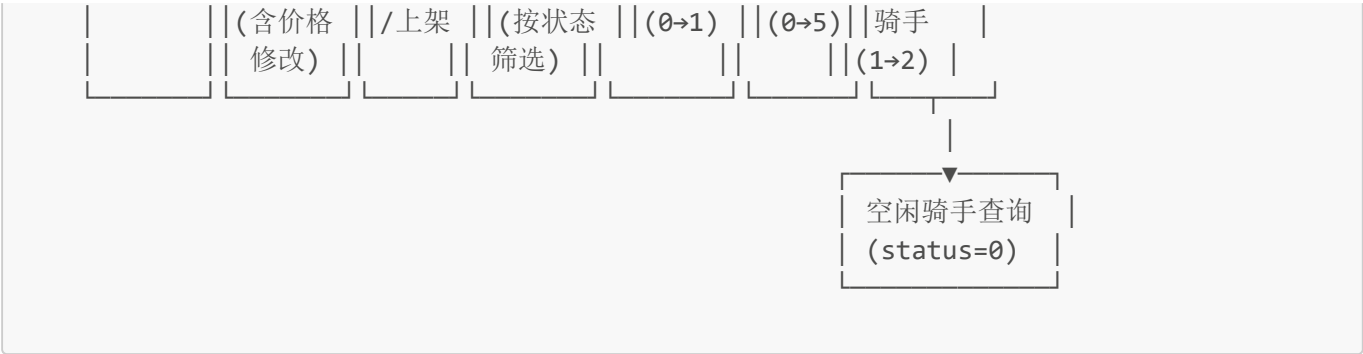
4.3 商家管理功能

4.3.1 功能概要

商家端提供菜品全生命周期管理（添加/编辑/软删除/上下架）和订单处理功能（查看/接单/拒单/指派骑手）。支持 WebSocket 实时接收新订单推送通知，并查询当前空闲骑手列表以完成派单。

用例图：





4.3.2 功能详细

菜品状态矩阵：

is_deleted	is_active	状态	顾客可见	商家可见
0	1	正常上架	✓ 可见	✓ 可见（可操作）
0	0	正常下架	✗ 不可见	✓ 可见（可重新上架）
1	1	已删除（上架）	✗ 不可见	✓ 可见（灰色，不可恢复）
1	0	已删除（下架）	✗ 不可见	✓ 可见（灰色，不可恢复）

订单处理流程：



代码实现位置：

组件	文件	说明
商家路由	<code>routers/merchant.py</code>	9 个 HTTP 端点 + 1 个 WebSocket 端点
WS 连接管理	<code>ws_manager.py</code>	<code>ConnectionManager</code> 类 (connect/disconnect/send_to_merchant/send_to_courier)

组件	文件	说明
商家页面	Merchant.vue	2 个 Tab：订单管理（含按状态筛选表格）+ 菜品管理（卡片网格）
API 调用	api/services.js	商家相关 9 个 API 封装函数

4.3.3 相关功能

组件名	说明	关系
WebSocket 管理器	实时接收新订单推送（new_order）；指派后通知骑手（order_assigned）	send_to_merchant() / send_to_courier()
骑手模块	骑手状态变更（空闲↔忙碌）与订单状态同步更新	指派时原子更新 courier.status
认证模块	商家角色鉴权，校验订单归属	Depends(get_current_merchant)

4.3.4 用户接口

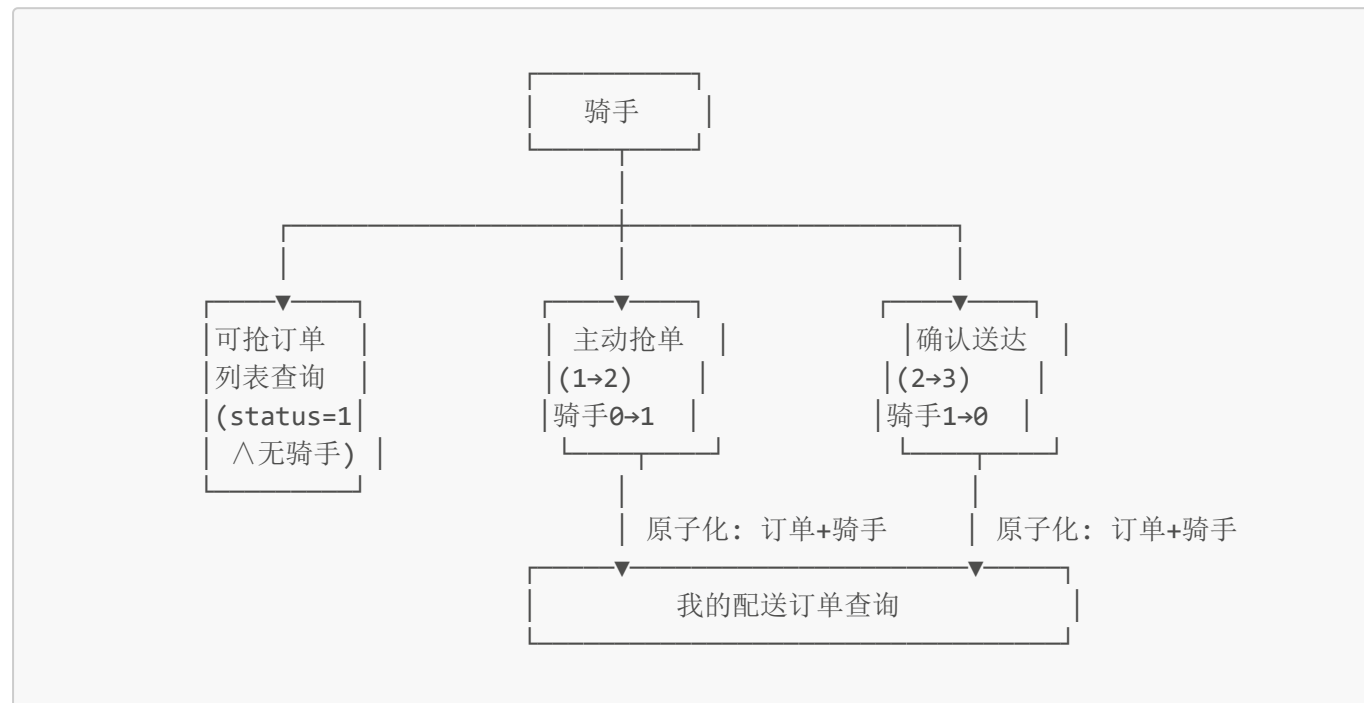
接口端点	方法	功能描述	鉴权角色
/api/merchant/dishes	GET	查看全部菜品列表（含已删除，方便恢复查看）	merchant
/api/merchant/dishes	POST	新增菜品	merchant
/api/merchant/dishes/{id}	PUT	更新菜品（名称/价格/上下架状态）	merchant
/api/merchant/dishes/{id}	DELETE	软删除菜品（is_deleted=1）	merchant
/api/merchant/orders	GET	查看该商家所有订单（最新在前，含菜品详情）	merchant
/api/merchant/order/{id}/accept	PUT	接单（0→1，校验仅待处理时可接单）	merchant
/api/merchant/order/{id}/reject	PUT	拒单（0→5，校验仅待处理时可拒单）	merchant
/api/merchant/order/{id}/assign	PUT	指派空闲骑手（1→2，原子更新订单+骑手状态）	merchant
/api/merchant/available-couriers	GET	查询所有空闲骑手列表（status=0，按名称排序）	merchant
/api/merchant/ws/{role}/{user_id}	WebSocket	实时消息接收（需 JWT Token 鉴权）	所有角色

4.4 骑手配送功能

4.4.1 功能概要

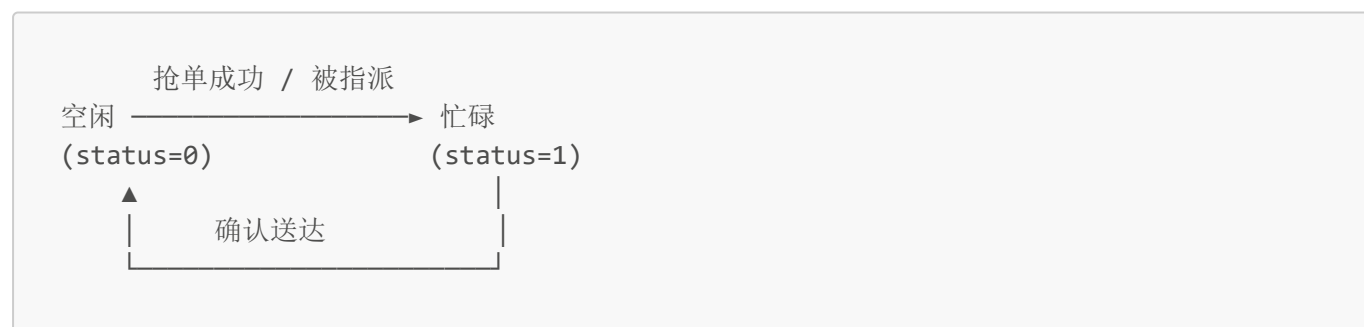
骑手端实现主动抢单和配送确认功能。骑手查看"商家已接单但无骑手配送 (status=1 且 courier_id IS NULL)"的订单列表，主动发起抢单；抢单成功后骑手状态变为"忙碌" (status=1)；配送完成后确认送达，骑手状态恢复为"空闲" (status=0)。

用例图：



4.4.2 功能详细

骑手状态机：



抢单流程：

骑手点击抢单 → 前端 PUT /api/courier/order/{id}/pickup



后端校验：

- ├─ [courier.status != 0] → 400 "您当前忙碌，无法接单"
- ├─ [订单不存在] → 404
- ├─ [order.status != 1] → 400 "该订单已被接单或不可抢"
- ├─ [order.courier_id IS NOT NULL] → 400 "该订单已被其他骑手抢走"
- └─ [全部通过]


```
▼
原子化操作：
  order.status = 2 (派送中)
  order.courier_id = courier.id
  courier.status = 1 (忙碌)
  order.order_time = now()
  |
  ▼
await db.commit()
  |
  ▼
返回 "抢单成功，请及时取餐派送"
```

代码实现位置：

组件	文件	说明
骑手路由	routers/courier.py	4 个端点：可抢订单列表、抢单、我的订单、确认送达
骑手页面	Courier.vue	2 个 Tab：可抢订单（卡片+抢单按钮）+ 我的派送（卡片+确认送达按钮）
API 调用	api/services.js	骑手相关 4 个 API 封装函数

4.4.3 相关功能

组件名	说明	关系
商家模块	商家接单后订单 status=1，骑手才能看到可抢订单	状态机联动（商家接单→可抢列表）
WebSocket 管理器	接收商家指派的实时通知	send_to_courier()
数据库层	空闲骑手查询视图 (get_available_couriers_query)	辅助商家查询

4.4.4 用户接口

接口端点	方法	功能描述	鉴权角色
/api/courier/available-orders	GET	查看可抢订单（status=1 且 courier_id IS NULL，按时间倒序）	courier
/api/courier/order/{id}/pickup	PUT	抢单（1→2，骑手 0→1，原子操作）	courier
/api/courier/orders	GET	查看我的配送订单（courier_id=当前骑手ID）	courier
/api/courier/order/{id}/deliver	PUT	确认送达（2→3，骑手 1→0，原子操作）	courier

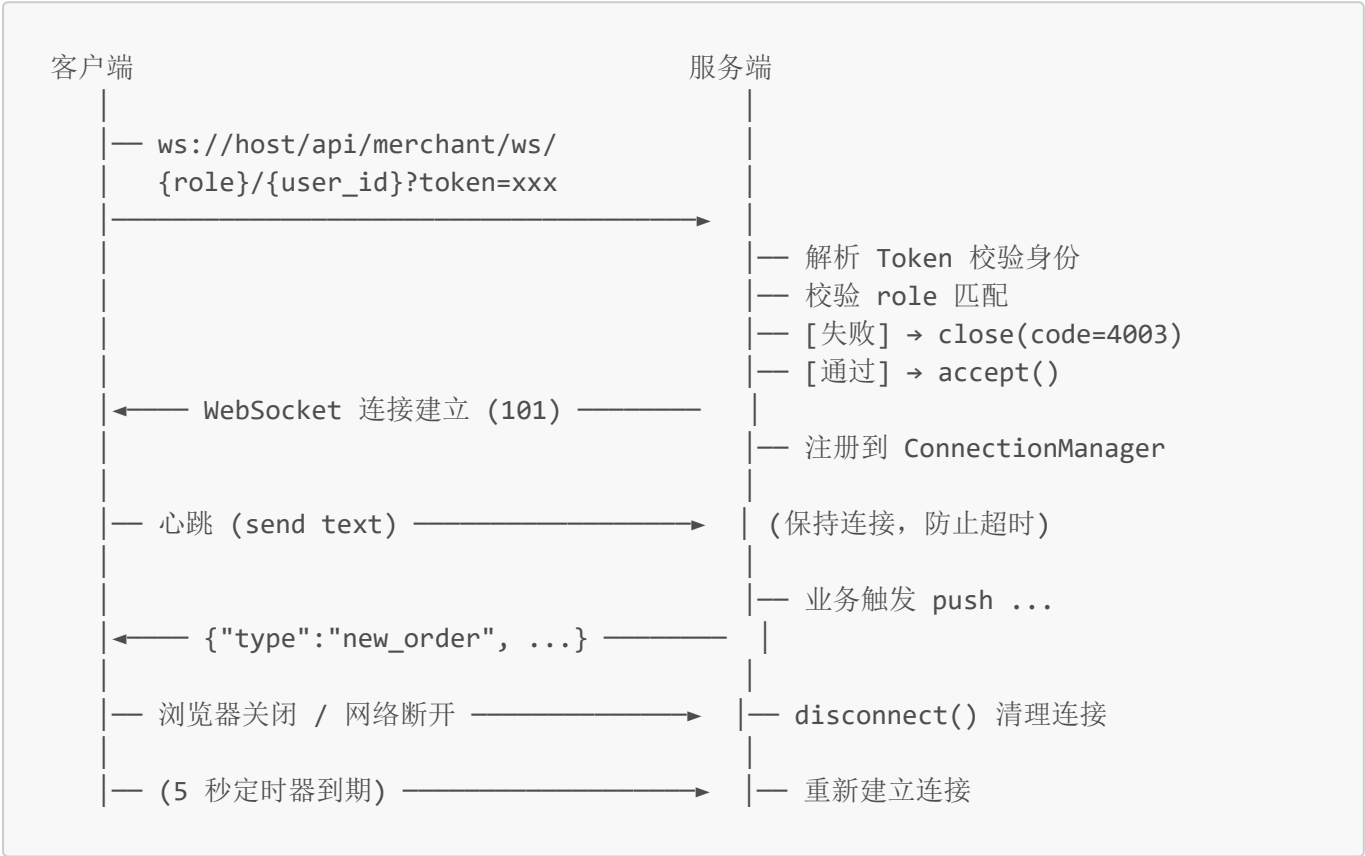
4.5 WebSocket实时推送功能

4.5.1 功能概要

WebSocket 模块为商家端和骑手端提供实时消息推送服务。当订单状态发生变化时（新订单、订单取消、骑手指派），服务端通过 WebSocket 长连接向目标用户实时推送 JSON 格式通知消息，用户无需手动刷新页面即可获得最新订单动态。

4.5.2 功能详细

连接生命周期：



ConnectionManager 设计：

```
_connections = {
  "merchant": { merchant_id: [ws1, ws2, ...] },
  "courier": { courier_id: [ws1, ws2, ...] },
  "customer": {}
}
```

- 按角色 + 用户 ID 两级索引管理连接，支持同一用户多设备同时连接
- 推送失败时静默跳过（try/except pass），不影响其他连接
- 断开时从对应 bucket 中移除

代码实现位置：

组件	文件	说明
WS 管理器	ws_manager.py	<code>ConnectionManager</code> 类（全局单例）
WS 端点	routers/merchant.py#L32-L59	WebSocket 连接入口
Token 解析	utils.py#L105-L110	<code>parse_token_optional()</code> 用于 WS 鉴权
前端 WS 连接	Merchant.vue#L204-L225	商家端 <code>connectWebSocket()</code>
前端 WS 连接	Courier.vue#L163-L184	骑手端 <code>connectWebSocket()</code>

4.5.3 相关功能

组件名	说明	关系
认证模块	WS 连接时的 Token 鉴权	<code>parse_token_optional(token)</code>
顾客路由	下单成功/取消订单后触发 WS 推送	<code>ws_manager.send_to_merchant()</code>
商家路由	指派骑手后触发 WS 推送	<code>ws_manager.send_to_courier()</code>

4.5.4 用户接口

接口端点	方法	功能描述	鉴权方式
<code>/api/merchant/ws/{role}/{user_id}?token=</code>	WebSocket	商家/骑手实时消息接收与心跳保持	JWT Token (Query String)

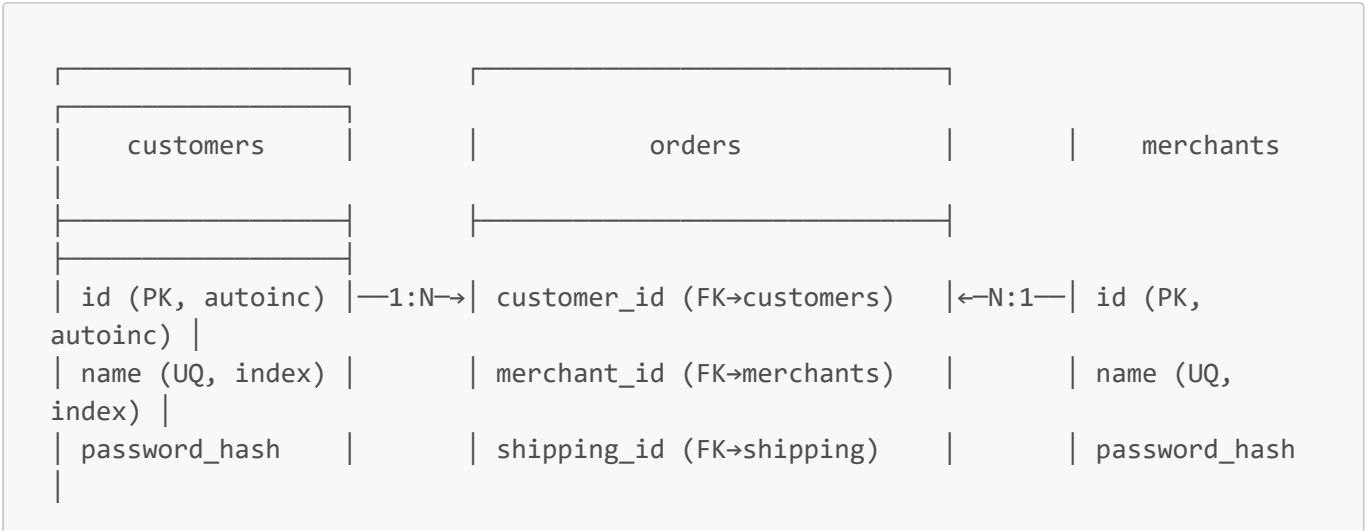
4.6 数据库层功能

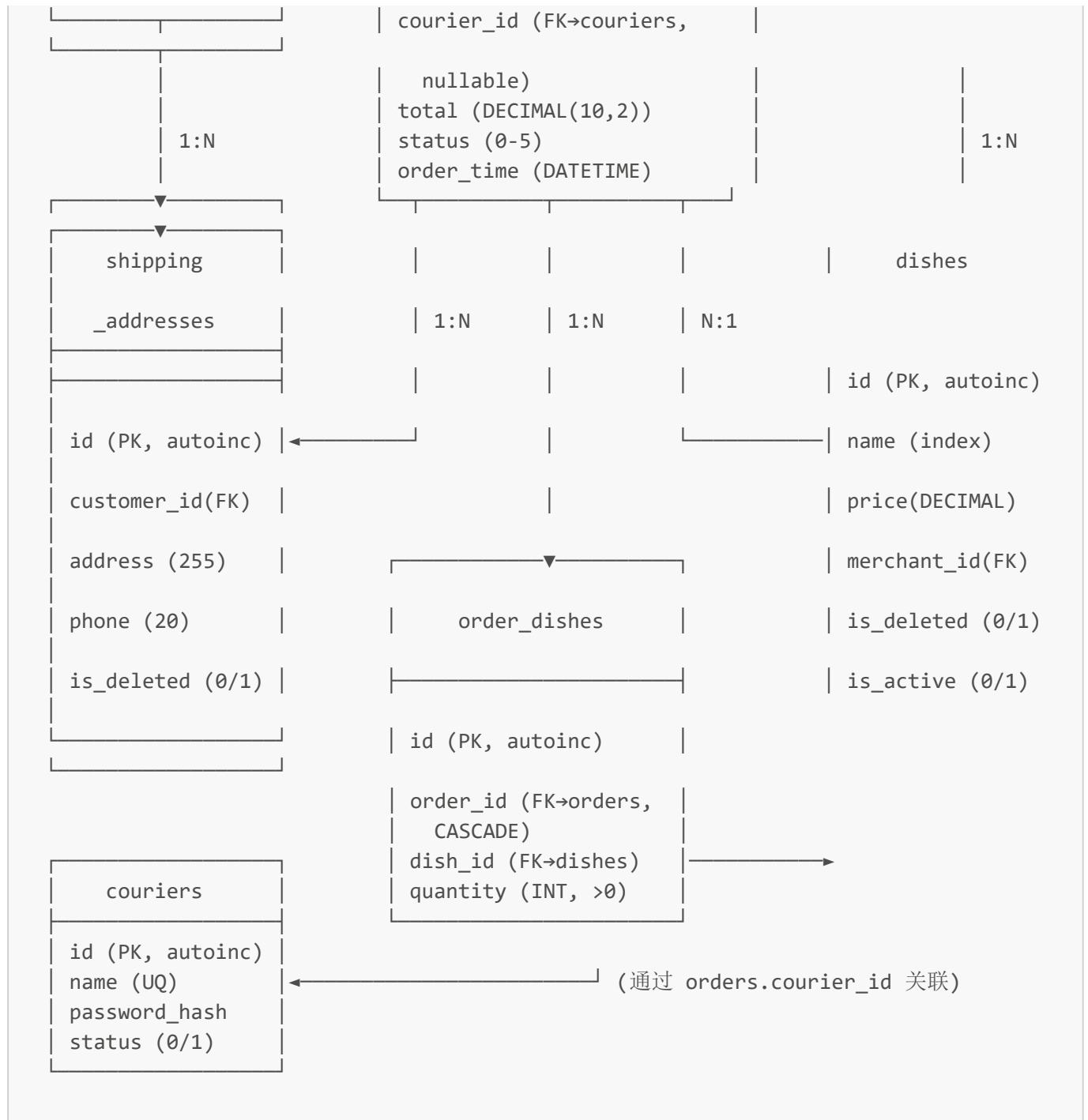
4.6.1 功能概要

数据库层采用 SQLAlchemy 2.0 异步 ORM 框架，共设计 7 张核心业务表，通过声明式模型映射数据库表结构。实现 ORM 事件触发器自动计算订单总额，以及空闲骑手查询辅助方法。

4.6.2 功能详细

7张核心业务表 ER 图：





表名	核心字段	约束与索引
customers	id(PK), name(UQ), password_hash	name 唯一索引
merchants	id(PK), name(UQ), password_hash	name 唯一索引
couriers	id(PK), name(UQ), password_hash, status	status: 0=空闲, 1=忙碌
dishes	id(PK), name, price, merchant_id(FK), is_deleted, is_active	CheckConstraint(price>=0), 复合索引 (merchant_id, is_deleted, is_active)
shipping_addresses	id(PK), customer_id(FK), address, phone, is_deleted	customer_id 索引

表名	核心字段	约束与索引
orders	id(PK), customer_id(FK), merchant_id(FK), shipping_id(FK), courier_id(FK, nullable), total(DECIMAL), status, order_time	四个外键，status: 0-5
order_dishes	id(PK), order_id(FK, CASCADE), dish_id(FK), quantity	CheckConstraint(quantity>0), order_id CASCADE 删除

ORM 事件触发器（总额自动计算）：

```
OrderDish 变更 (INSERT / UPDATE / DELETE)
├── SQLAlchemy ORM Event 触发
│   └── _recalc_order_total(connection, order_id)
│       ├── SELECT SUM(order_dishes.quantity × dishes.price)
│       │   FROM order_dishes
│       │   JOIN dishes ON order_dishes.dish_id = dishes.id
│       │   WHERE order_dishes.order_id = ?
│       └── UPDATE orders SET total = <result> WHERE orders.id = ?
```

触发器代码位置：[models.py#L189-L201](#) — `_od_after_insert`、`_od_after_update`、`_od_after_delete`

空闲骑手查询：

[models.py#L207-L213](#) — `get_available_couriers_query()` 返回 `SELECT * FROM couriers WHERE status = 0 ORDER BY name`

4.6.3 相关功能

组件名	说明	关系
所有路由模块	所有数据读写均通过 SQLAlchemy ORM AsyncSession 操作	依赖 <code>get_db()</code> 依赖注入获取会话
config.py	数据库连接 URL 配置、引擎参数	<code>settings.DATABASE_URL</code>
database.py	引擎初始化（SQLite/MySQL 自适应）、会话工厂	应用启动时通过 lifespan 创建表

4.6.4 用户接口

数据库层不直接面向用户，通过 ORM 向服务层提供数据访问能力。开发阶段数据库表在应用启动时通过 `Base.metadata.create_all` 自动创建（[main.py#L18](#)）。

5 非功能需求

5.1 权限需求

系统采用 RBAC（Role-Based Access Control）权限模型，基于 JWT Token 中的 `role` 字段进行权限控制。

用户角色	可访问的页面路由	可调用的 API 前缀	权限边界
customer	/customer	/api/auth/, /api/customer/	仅可操作本人订单和地址（数据归属校验）
merchant	/merchant	/api/auth/, /api/merchant/	仅可操作本店菜品和订单（数据归属校验）
courier	/courier	/api/auth/, /api/courier/	仅可操作本人配送订单

权限控制三层实现：

层级	实现方式	代码位置
后端 API 层	每个端点通过 <code>Depends(get_current_xxx)</code> 声明角色依赖，Token 角色不匹配返回 403	utils.py#L53-L101
前端路由层	<code>beforeEach</code> 路由守卫校验 <code>meta.role</code> 与 Pinia auth store 中的 <code>role</code> 是否一致	router/index.js#L36-L58
WebSocket 层	连接时通过 <code>parse_token_optional()</code> 解析 Token 并校验 <code>role</code> 与 <code>user_id</code> 匹配	merchant.py#L44-L47

5.2 兼容互换性

【没有特别要求】

本系统为全新开发项目，不涉及对旧版本系统的兼容性需求。向下兼容性不是本次开发的范围。

5.3 安全性

安全措施	实现方式	代码位置
密码加密存储	bcrypt 单向哈希（passlib CryptContext），不存储明文密码	utils.py#L19-L27
JWT 无状态认证	HS256 算法签名，payload 含 <code>user_id/role/name/exp</code> ，24 小时过期	utils.py#L31-L40
越权访问防护	后端：API 层通过角色依赖注入 + 数据归属校验（如校验订单是否属于当前用户）；前端：路由守卫角色校验	utils.py#L53-L101
JWT 密钥外部化	通过 <code>config.py</code> 的 <code>JWT_SECRET</code> 配置项注入，可经 <code>.env</code> 文件或环境变量覆盖	config.py#L17
CORS 白名单	仅允许配置在 <code>CORS_ORIGINS</code> 列表中的域名进行跨域访问	config.py#L22-L25 + main.py#L32-L38
前端 Token 管理	Token 存储在 localStorage，Axios 响应拦截器在 401 时自动清除 Token 并重定向到登录页	api/index.js#L35-L39

安全措施	实现方式	代码位置
路由级安全	未登录用户无法访问任何业务页面（自动重定向到登录页）； 已登录用户无法访问其他角色的页面	router/index.js#L36-L58
SQL 注入防护	全部数据库操作通过 SQLAlchemy ORM 参数化查询，严禁手写拼接 SQL	全项目 ORM 使用 select() 参数化

5.4 健壮性

措施	实现说明
事务原子性	下单操作使用 <code>async with db.begin()</code> 显式事务边界，任何校验失败抛出 <code>HTTPException</code> （自动 ROLLBACK），避免产生僵尸订单
WebSocket 自动重连	前端 <code>ws.onclose</code> 事件触发 5 秒后自动重连，网络短暂中断自动恢复
HTTPException 透传	下单接口中 <code>HTTPException</code> 异常直接透传（不触发通用的 500 处理），确保业务校验错误能准确返回客户端
数据库连接池	MySQL 模式下配置 <code>pool_size=20</code> 、 <code>max_overflow=10</code> 、 <code>pool_pre_ping=True</code> ，防止连接池耗尽和失效连接复用
SQLite 特殊处理	SQLite 模式下设置 <code>check_same_thread=False</code> ，适配异步场景
软删除机制	菜品和地址采用软删除（ <code>is_deleted=1</code> ），删除操作仅修改标记位，历史订单的外键引用不会断裂
前端错误兜底	所有 API 调用使用 <code>try/catch</code> 包裹， <code>catch</code> 块静默处理（错误已在 Axios 拦截器中通过 <code>EIMessage</code> 提示）

容错等级：

- **成熟性**：核心 API 经过 22 个端点逐一测试，常规操作无预期故障
- **障害兼容性**：单个 API 异常不影响其他功能模块，前端 `catch` 块保证页面不崩溃
- **恢复性**：SQLite 文件数据库无需额外恢复操作；MySQL 模式下连接池自动重连

5.5 使用性（操作性）

方面	设计说明
三端统一登录页	同一页面通过角色选择按钮（顾客/商家/骑手）支持三种角色登录，切换角色自动清空表单，登录/注册状态一键切换
响应式设计	顾客端和骑手端： <code>max-width: 600px</code> 居中布局（模拟移动端体验）；商家端： <code>max-width: 1100px</code> 宽表格布局（PC 工作台体验）
实时反馈	商家端和骑手端通过 WebSocket 实时接收推送通知（ <code>EINotification</code> 弹窗 + 新订单计数 badge）
表单双重校验	前端：Element Plus 表单规则验证（必填、长度等）；后端：Pydantic 数据校验（类型、范围、正则匹配）

方面	设计说明
友好错误提示	错误响应通过 Element Plus Message 组件展示后端返回的具体原因（如"当前订单状态为「已接单」，无法取消"）
操作确认	关键操作（取消订单、确认抢单、确认送达、删除地址、删除菜品、切换商家）均弹出 ElMessageBox 确认对话框
购物车体验	点击菜品自动加入购物车（带 + 按钮动画），购物车数量实时显示 badge，切换商家自动清空购物车并提示
视觉层次	订单按状态以不同颜色的左边框区分（待处理/已接单/派送中/已完成/已取消/已拒单），一看看出订单状态

5.6 效率性（性能）

指标	目标值	说明
API 响应时间	< 500ms（常规 CRUD）	开发环境 SQLite + 本地单用户基准
WebSocket 推送延迟	< 1 秒	下单 → 商家收到 new_order 推送
并发用户数	100 用户同时操作	适用中小型商户场景
数据库连接池	pool_size=20, max_overflow=10	仅 MySQL 模式启用连接池（SQLite 不适用）
前端首屏加载	Vite + Element Plus 全量引入	首包约 1.2MB（未做按需加载）
订单查询	按 order_time DESC 查询，带 ORM relationship 预加载	lazy="selectin" 减少 N+1 查询
菜品浏览查询	复合索引 idx_dish_merchant_active	覆盖最频繁的查询条件组合

性能优化措施：

- 所有 ORM relationship 使用 lazy="selectin" 策略，避免 N+1 查询问题
- dishes 表建立复合索引 (merchant_id, is_deleted, is_active) 加速商家菜品查询
- customers 和 merchants 表 name 字段建立唯一索引，加速登录查询
- 订单查询一次加载所有关联数据（customer/merchant/courier/shipping/order_dishes/dish）

5.7 维护性

措施	实现说明
模块化架构	后端按功能垂直拆分为 auth/customer/merchant/courier 四个路由模块；前端按视图拆分为四个页面组件
ORM 抽象层	全部数据库操作通过 SQLAlchemy ORM 实现，绝不手写拼接 SQL，数据库切换时无需修改业务代码

措施	实现说明
共享工具函数	<code>build_order_out()</code> 在 <code>service_utils.py</code> 中实现，供顾客/商家/骑手三端路由模块复用，修改一处即全局生效
API 文档自动生成	FastAPI 内置 Swagger UI (<code>/docs</code>) 和 ReDoc，接口代码变更自动同步文档，无需额外维护
常量集中管理	<code>STATUS_TEXT</code> 状态码到中文的映射定义在 <code>service_utils.py</code> 中，前后端状态文本统一维护
配置外部化	数据库 URL、JWT 密钥、CORS 白名单等通过 <code>config.py</code> 集中管理，支持 <code>.env</code> 文件覆盖
错误定位便捷	后端 <code>HTTPException</code> 携带具体的错误描述（如"菜品「黄金炒饭」已下架"），前端直接展示，排查问题效率高

分析性： 每个 API 端点的职责单一、命名自解释，配合 Swagger UI 可快速定位问题接口。ORM 事件触发器的日志可通过设置 `echo=True` 开启 SQL 调试输出。

5.8 移植性

方面	说明
数据库迁移	开发环境使用 SQLite（零配置），生产环境切换 MySQL 仅需修改 <code>DATABASE_URL</code> 配置项，ORM 层自动适配方言差异
OS 兼容性	Python 3.10+ + Node.js 18+ 跨平台，源码无需修改即可在 Windows / macOS / Linux 上运行
前端部署灵活性	生产构建产物（ <code>npm run build → dist/</code> ）为纯静态 HTML/CSS/JS 文件，可部署于 Nginx / Apache / Caddy 等任意 Web 服务器
数据库方言适配	使用 SQLAlchemy ORM 抽象层，不依赖特定数据库方言； <code>database.py</code> 中针对 SQLite 做了 <code>check_same_thread</code> 特殊处理，其他数据库使用通用连接池配置

移植策略： 全程不依赖操作系统特有 API 或文件系统特性；密码哈希使用纯 Python 实现的 `bcrypt`（`passlib` 库），不依赖系统级加密模块。

5.9 用户文档

文档	交付格式	说明
项目开发计划书	Markdown	项目整体计划、人员分工、技术方案、开发日程、风险管理
需求与功能设计规格书（本文档）	Markdown	功能模块详细设计、接口定义、数据字典、业务规则
部署说明文档	Markdown (README.md)	环境要求、安装步骤、配置说明、首次使用指南、常见问题 FAQ
API 接口文档	Swagger UI (<code>/docs</code>)	22 个 API 端点在线文档，支持交互式测试
数据库设计文档	<code>models.py</code> + 本文档附录	7 张表 ORM 模型定义 + ER 图 + 数据字典

5.10 其他

【没有特别记述的事项】

6 使用和操作方法

6.1 环境设定

6.1.1 后端配置文件（backend/app/config.py）

配置项	默认值	说明
DATABASE_URL	sqlite+aiosqlite:///./takeaway.db	开发环境使用 SQLite 零配置；生产环境改为 MySQL 连接串
JWT_SECRET	默认值	生产环境务必修改为随机 64 位字符串
JWT_ALGORITHM	HS256	JWT 签名算法，标准值无需修改
JWT_EXPIRE_MINUTES	1440（24 小时）	Token 过期时间，可根据安全需求调整
CORS_ORIGINS	["http://localhost:5173"]	允许跨域的前端地址列表
APP_TITLE	外卖管理系统 API	API 文档标题
APP_VERSION	1.0.0	应用版本号

通过 .env 文件覆盖（推荐生产环境使用）：

```
DATABASE_URL=mysql+asyncmy://user:pass@127.0.0.1:3306/takeaway
JWT_SECRET=your-production-random-secret-key
CORS_ORIGINS=["https://your-domain.com"]
```

6.1.2 前端配置（frontend/vite.config.js）

- Vite 开发服务器默认端口：5173
- API 代理配置：/api 请求自动转发到 http://127.0.0.1:8001

6.1.3 VS Code 配置文件（.vscode/settings.json）

- 隐藏 node_modules 目录，提升文件浏览性能

6.2 使用方法

6.2.1 安装依赖

后端：

```
cd backend
pip install -r requirements.txt
```

前端：

```
cd frontend
npm install
```

6.2.2 启动服务

终端 1 — 启动后端：

```
cd backend
uvicorn app.main:app --host 0.0.0.0 --port 8001 --reload
```

启动成功：显示 `Uvicorn running on http://0.0.0.0:8001` 和 `Application startup complete.`

终端 2 — 启动前端：

```
cd frontend
npm run dev
```

启动成功：显示 `VITE v5.x.x ready in xxx ms` 和 `Local: http://localhost:5173/`

6.2.3 验证服务

浏览器访问 `http://127.0.0.1:8001/` 显示：

```
{"status": "ok", "service": "外卖管理系统 API", "version": "1.0.0"}
```

API 交互式文档：`http://127.0.0.1:8001/docs`

6.2.4 首次使用流程

1. 访问 `http://localhost:5173`，进入登录页
2. 注册三个角色账户（建议用户名：m1 商家 / u1 顾客 / c1 骑手）
3. 商家登录 → 菜品管理 → 添加菜品
4. 顾客登录 → 选择商家 → 浏览菜单 → 加入购物车 → 添加地址 → 下单
5. 商家收到新订单 WebSocket 推送 → 接单
6. 骑手登录 → 可抢订单 → 抢单 → 确认送达
7. 全流程完成：订单状态 0→1→2→3

6.2.5 退出系统

- 后端：终端中按 **Ctrl+C** 停止 uvicorn 服务器
- 前端：终端中按 **Ctrl+C** 停止 Vite 开发服务器
- 用户退出登录：点击页面右上角「退出」按钮，清除 Token 并返回登录页

 提示：不同角色请使用不同的浏览器或无痕窗口登录，避免 localStorage 中的 Token 互相覆盖。

7 注意限制事项

7.1 制约／限制事项

编号	限制事项	详细说明
LN-01	单商家单骑手模型	当前版本每个订单仅由一个商家处理、一个骑手配送，不支持多商家拼单或多骑手接力配送
LN-02	无支付集成	当前版本不包含在线支付功能，订单金额仅为展示用途，不涉及真实资金流转
LN-03	无图片上传	菜品不支持图片展示，仅显示名称和价格
LN-04	无地理位置追踪	骑手不支持实时位置追踪和距离计算
LN-05	骑手抢单无分布式锁	高并发场景下可能存在竞态条件（同一订单被多个骑手同时抢到），当前版本通过数据库约束作为最后防线
LN-06	浏览器兼容性	建议使用 Chrome/Edge 最新版本，不支持 Internet Explorer
LN-07	无分页机制	订单列表和菜品列表未实现分页，数据量大时可能存在性能问题
LN-08	单服务器部署	系统设计为单实例部署，不支持分布式集群（WebSocket 状态存储在进程内存中）
LN-09	数据库文件存储	开发环境 SQLite 数据库文件存储在项目目录下，需注意备份

7.2 假定事项

编号	假定事项	详细说明
AS-01	开发阶段使用 SQLite	假定开发阶段无需安装配置 MySQL，使用 SQLite 零配置数据库
AS-02	用户规模	假定系统服务于中小型餐饮商家，并发用户在 100 以内

编号	假定事项	详细说明
AS-03	单实例部署	假定系统部署在单台服务器上，不涉及分布式集群架构
AS-04	开发工具	假定开发人员使用 VS Code + Windows 10/11 环境
AS-05	网络环境	假定用户在局域网或稳定的互联网环境中使用系统
AS-06	密码强度	假定用户使用至少 4 位密码，系统不做密码复杂度强制校验
AS-07	一人一角色	假定每个用户只注册一种角色，同一用户名在不同角色表中可以重复（实际因三表独立可以存在）

7.3 特记事项

【无特记事项】

8 附录

8.1 业务规则

编号	规则描述	涉及角色	代码位置
BR-01	订单仅在 status=0（待处理）时可被顾客取消（→4）	顾客	customer.py#L288
BR-02	订单仅在 status=0（待处理）时可被商家接单（→1）	商家	merchant.py#L176
BR-03	订单仅在 status=0（待处理）时可被商家拒单（→5）	商家	merchant.py#L195
BR-04	仅空闲骑手（status=0）可抢单	骑手	courier.py#L57
BR-05	仅已接单（status=1）且无骑手（courier_id IS NULL）的订单可被抢	骑手	courier.py#L63-L65
BR-06	仅已接单（status=1）的订单商家可指派骑手	商家	merchant.py#L226
BR-07	仅派送中（status=2）且分配给自己的订单骑手可确认送达（→3）	骑手	courier.py#L114-L116
BR-08	商家指派骑手时需校验骑手空闲（status=0）	商家	merchant.py#L232

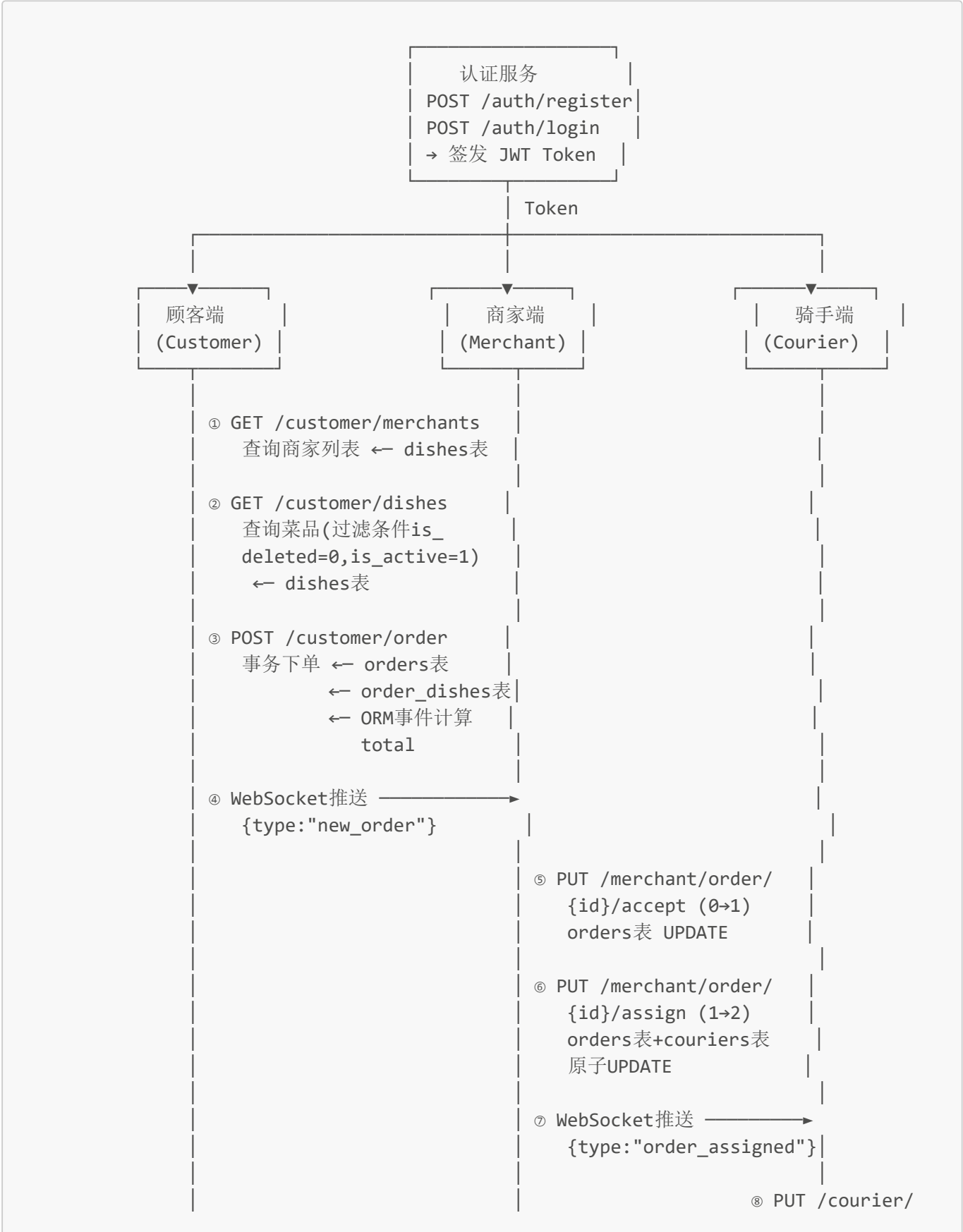
编号	规则描述	涉及角色	代码位置
BR-09	顾客只能操作自己的订单（customer_id 匹配）和地址	顾客	customer.py#L162,L286
BR-10	商家只能操作自己店铺的菜品和订单（merchant_id 匹配）	商家	merchant.py#L115,L174
BR-11	骑手只能确认送达分配给自己的订单（courier_id 匹配）	骑手	courier.py#L114
BR-12	同一用户名不可在同一角色表中重复注册（name UNIQUE 约束）	全部	auth.py#L33
BR-13	菜品删除为软删除（is_deleted=1），数据物理保留	商家	merchant.py#L143
BR-14	地址删除为软删除（is_deleted=1），不影响历史订单	顾客	customer.py#L131
BR-15	顾客下单时所有菜品必须属于同一个指定商家	顾客	customer.py#L194
BR-16	订单总额由 ORM 触发器自动计算（SUM(price × quantity)），不允许手动修改	系统	models.py#L189-L201

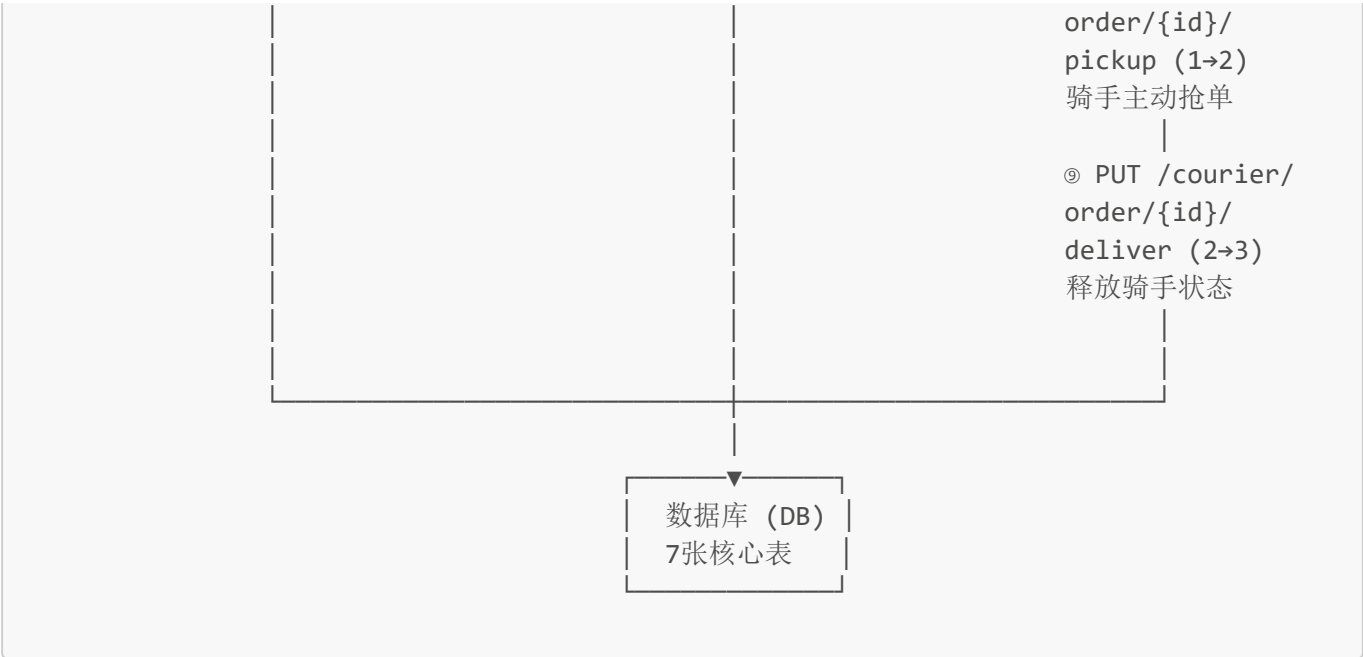
8.2 术语表

术语	英文	说明
订单状态机	Order State Machine	订单状态有序流转规则：0待处理→1已接单→2派送中→3已完成；分支：0→4顾客已取消、0→5商家已拒单
软删除	Soft Delete	通过标记字段（is_deleted=1）而非物理删除方式移除数据，保留数据用于历史追溯
原子事务	Atomic Transaction	数据库事务保证多个操作要么全部成功提交（COMMIT），要么全部回滚（ROLLBACK）
WebSocket 推送	WebSocket Push	服务端通过 TCP 长连接主动向指定客户端实时推送 JSON 消息
JWT Token	JSON Web Token	包含 user_id、role、name、过期时间的签名令牌，用于无状态客户端认证
RBAC	Role-Based Access Control	基于角色的访问控制：通过 payload 中的 role 字段决定用户可访问的 API 范围
ORM 事件	ORM Event	SQLAlchemy 提供的对象关系映射级事件钩子，在 INSERT/UPDATE/DELETE 时触发自定义回调
依赖注入	Dependency Injection	FastAPI 的 Depends 机制，用于自动注入数据库会话和认证用户实例

术语	英文	说明
复合索引	Composite Index	多列组合的数据库索引，加速多条件查询
连接池	Connection Pool	预创建并维护一组数据库连接，复用连接减少开销

8.3 数据流图





数据流步骤说明：

步骤	数据流向	说明
①	顾客端→数据库→顾客端	查询有在售菜品的商家列表（聚合 dishes 表统计菜品数量）
②	顾客端→数据库→顾客端	查询指定商家的在售菜品（is_deleted=0, is_active=1）
③	顾客端→数据库	事务写入 orders + order_dishes，ORM 事件触发总额计算
④	后端→商家端（WS）	下单成功后 WebSocket 推送 new_order 提醒
⑤	商家端→数据库	更新 orders.status=1（接单）
⑥	商家端→数据库	原子更新 orders（status+ courier_id）+ couriers（status）
⑦	后端→骑手端（WS）	指派骑手后 WebSocket 推送 order_assigned
⑧	骑手端→数据库	主动抢单，原子更新 orders + couriers
⑨	骑手端→数据库	确认送达，orders.status=3，couriers.status=0

8.4 数据字典

customers（顾客表）

属性	类型	说明	取值范围	备注
id	INTEGER	主键	自增正整数	PRIMARY KEY, AUTOINCREMENT
name	VARCHAR(50)	用户名	任意字符，最长 50	UNIQUE, NOT NULL, INDEX
password_hash	VARCHAR(255)	bcrypt 密码哈希		NOT NULL

merchants（商家表）

属性	类型	说明	取值范围	备注
id	INTEGER	主键	自增正整数	PRIMARY KEY, AUTOINCREMENT
name	VARCHAR(50)	商家名称	任意字符，最长 50	UNIQUE, NOT NULL, INDEX
password_hash	VARCHAR(255)	bcrypt 密码哈希		NOT NULL

couriers（骑手表）

属性	类型	说明	取值范围	备注
id	INTEGER	主键	自增正整数	PRIMARY KEY, AUTOINCREMENT
name	VARCHAR(50)	骑手名称	任意字符，最长 50	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	bcrypt 密码哈希		NOT NULL
status	INTEGER	骑手忙碌状态	0: 空闲, 1: 忙碌	NOT NULL, DEFAULT 0

dishes（菜品表）

属性	类型	说明	取值范围	备注
id	INTEGER	主键	自增正整数	PRIMARY KEY, AUTOINCREMENT
name	VARCHAR(100)	菜品名称	任意字符，最长 100	NOT NULL, INDEX
price	DECIMAL(10,2)	单价（元）	≥ 0	NOT NULL, CheckConstraint("price >= 0")
merchant_id	INTEGER	所属商家 ID	有效的 merchants.id	FOREIGN KEY, ON DELETE CASCADE
is_deleted	INTEGER	软删除标记	0: 正常, 1: 已删除	NOT NULL, DEFAULT 0
is_active	INTEGER	上下架状态	0: 已下架, 1: 已上架	NOT NULL, DEFAULT 1
复合索引		idx_dish_merchant_active	(merchant_id, is_deleted,	加速商家菜品查询

属性	类型	说明	取值范围	备注
			is_active)	

shipping_addresses（收货地址表）

属性	类型	说明	取值范围	备注
id	INTEGER	主键	自增正整数	PRIMARY KEY, AUTOINCREMENT
customer_id	INTEGER	所属顾客ID	有效的 customers.id	FOREIGN KEY, ON DELETE CASCADE, INDEX
address	VARCHAR(255)	详细地址文本	任意字符，最长255	NOT NULL
phone	VARCHAR(20)	联系电话	数字及符号，最长20	NOT NULL
is_deleted	INTEGER	软删除标记	0: 正常, 1: 已删除	NOT NULL, DEFAULT 0

orders（订单主表）

属性	类型	说明	取值范围	备注
id	INTEGER	主键	自增正整数	PRIMARY KEY, AUTOINCREMENT
customer_id	INTEGER	下单顾客ID	有效的 customers.id	FOREIGN KEY, NOT NULL
merchant_id	INTEGER	接单商家ID	有效的 merchants.id	FOREIGN KEY, NOT NULL
shipping_id	INTEGER	收货地址ID	有效的 shipping_addresses.id	FOREIGN KEY, NOT NULL
courier_id	INTEGER	配送骑手ID	有效的 couriers.id 或 NULL	FOREIGN KEY, NULLABLE（初始为空）
total	DECIMAL(10,2)	订单总额	≥ 0	NOT NULL, DEFAULT 0；由 ORM 事件自动计算
status	INTEGER	订单状态码	0-5	NOT NULL, DEFAULT 0
order_time	DATETIME	最后一次状态变更时间		NOT NULL, DEFAULT NOW

订单状态码含义：

值	状态名	说明	下一有效状态
0	待处理	顾客刚下单，等待商家处理	1（商家接单）、4（顾客取消）、5（商家拒单）
1	已接单	商家已确认订单，等待骑手接单	2（骑手抢单/商家指派）
2	派送中	骑手已接单，正在配送途中	3（骑手确认送达）
3	已完成	订单配送完成，终态	无
4	客户已取消	顾客在待处理状态下取消，终态	无
5	商家已拒单	商家拒绝该订单，终态	无

order_dishes（订单-菜品关联表）

属性	类型	说明	取值范围	备注
id	INTEGER	主键	自增正整数	PRIMARY KEY, AUTOINCREMENT
order_id	INTEGER	所属订单 ID	有效的 orders.id	FOREIGN KEY, ON DELETE CASCADE
dish_id	INTEGER	菜品 ID	有效的 dishes.id	FOREIGN KEY
quantity	INTEGER	购买数量	> 0	NOT NULL, CheckConstraint("quantity > 0")

注：order_dishes 表是 orders 与 dishes 之间的多对多中间表，一条订单可有多种菜品，每种菜品有一个数量。ORDERS 级联删除时（CASCADE），关联的 OrderDish 记录自动删除。OrderDish 的任何 INSERT/UPDATE/DELETE 操作都会触发 ORM 事件，自动重新计算所属订单的 total 字段。

文档状态：已发布

最后更新：2026年6月9日